

ITS SOFTWARE ARCHITECT SPECIALIST
2023-2025

Z E U S

SOFTWARE DI CONTROLLO PER ALIMENTATORI
COMPATTI DI ALTA TENSIONE

L U C A R I G A M O N T I



ITS ACADEMY
ANGELO RIZZOL



Tecnico Superiore per lo sviluppo di sistemi full stack per web,
mobile e desktop

ITS Software Architect Specialist

NUOVA EDIZIONE

Biennio 2023-25

ZEUS

Software di controllo per alimentatori compatti di alta tensione

Luca Rigamonti

*“ Did you have a good world when you died?
Enough to base a movie on? “*

- Jim Morrison

INDICE DEI CONTENUTI

I. PRESENTAZIONE

<i>I.I - Chi sono</i>	7
<i>I.II - Percorso di studi</i>	8

II. NUCLEAR INSTRUMENTS

<i>II.I – Introduzione</i>	10
<i>II.II – Tecnologie offerte</i>	12
<i>II.III – Partnership</i>	15
<i>II.IV – Ambiente lavorativo</i>	17

III. IL MIO RUOLO NELL’AZIENDA

<i>III.I Responsabilità e contributi tecnici</i>	18
--	-----------

IV. ZEUS

<i>IV.I – Descrizione generale</i>	21
<i>IV.II – Il primo ZEUS</i>	22
<i>IV.III – Requisiti principali</i>	25
IV.IV – Sviluppo di ZEUS	28
<i>IV.IV.I – Ambiente di sviluppo</i>	28
<i>IV.IV.I.I – Visual Studio Code</i>	29
<i>IV.IV.I.I.I – Strumenti di supporto</i>	30
<i>IV.IV.I.I.I.I – Serial Monitor</i>	30
IV.IV.II – Tecnologie utilizzate	32
<i>IV.IV.II.I – Wails</i>	32
<i>IV.IV.II.I.I – Go</i>	34
<i>IV.IV.II.I.II – React</i>	35
<i>IV.IV.II.I.II.I – LightningChart</i>	36

INDICE DEI CONTENUTI

IV.IV.III – Sistema modulare	38
IV.IV.IV – Architettura del backend	39
IV.IV.V – Architettura del frontend	46
IV.IV.VI – Comunicazione tra backend e frontend	53
IV.IV.VII – Sezioni di ZEUS	57
IV.IV.VII.I – Tabella di controllo	57
IV.IV.VII.I.I – Sidebar Devices	59
IV.IV.VII.I.II – Tabella principale	64
IV.IV.VII.I.III – Sidebar Channel	71
IV.IV.VII.II – Plot in tempo reale	76
IV.IV.VII.II.I – Sidebar Channels	77
IV.IV.VII.II.II – Plot	80
IV.IV.VII.II.II.I – Time Plot	80
IV.IV.VII.II.II.II – Histogram Plot	84
IV.IV.VII.II.III – Registrazione su file	87
IV.IV.VII.II.III.I – Accesso al file system	95
IV.IV.VII.III – Curva I-V	97
IV.IV.VII.III.I – Sidebar settings	98
IV.IV.VII.III.II – Plot IV	101
V – CONCLUSIONI	
V.I – Difficoltà incontrate	103
V.II – Competenze ottenute	104
V.III – Aspettative e realtà	106

INDICE DEI CONTENUTI

 VI. SITOGRAFIA

108

VII. RINGRAZIAMENTI

109

I. PRESENTAZIONE

I.1 - Chi sono

Mi chiamo Luca Rigamonti, ho 20 anni. Sono nato a Milano, ma sono cresciuto a Muggiò, una città in provincia di Monza e Brianza.

Fin da piccolo mi sono sempre considerato una persona creativa e curiosa, ma allo stesso tempo calma e riflessiva. Questo approccio mi ha aiutato ad affrontare e risolvere gli ostacoli che si sono presentati lungo il mio percorso.

Ho sempre avuto le idee chiare sul mio futuro, cercando di apprendere quanto più possibile dalle mie esperienze formative, lavorative e umane.

Le mie più grandi passioni sono la musica e la tecnologia. In entrambe le aree, cerco sempre di realizzare soluzioni creative che mi stimolino ogni volta a dare il massimo.

I.II - Percorso di studi

Il mio percorso di studi si è sviluppato lungo due direzioni principali: la formazione tecnica in ambito informatico e la passione per la musica, che ho sempre portato avanti in parallelo.

Ho conseguito il Diploma di Istituto Tecnico nel settore tecnologico, indirizzo “Informatica e Telecomunicazioni”, presso l’Istituto P. Henseberger di Monza. Qui ho appreso i principi fondamentali dell’informatica, le basi del polimorfismo, la programmazione orientata agli oggetti e le basi della programmazione web. Questo mi ha permesso di sviluppare un solido approccio analitico e una comprensione strutturata dello sviluppo software.

Successivamente, ho scelto di proseguire la mia formazione presso l’ITS Academy – Istituto Tecnologico Superiore Angelo Rizzoli di Milano, frequentando il corso Software Architect Specialist a Seregno, per diventare un Tecnico Superiore per lo sviluppo di sistemi full stack per web, mobile e desktop. Ho scelto questo percorso perché, terminate le superiori, ritenevo che le mie conoscenze e capacità non fossero ancora sufficienti per intraprendere una carriera lavorativa.

In questo corso ho approfondito in modo più avanzato le conoscenze acquisite alle superiori, portandole a un livello professionale e applicativo.

Parallelamente agli studi informatici, ho sempre coltivato la mia passione per la musica, conseguendo nel 2024 la Certificazione di Strumento Batteria presso l'Accademia Musicale Italiana di Monza.

Allo stesso tempo, ho sempre suonato in una band, esibendomi su numerosi palchi, tra cui alcuni dei più importanti della scena milanese.

Questo percorso musicale ha rappresentato per me una fonte costante di ispirazione e creatività, che cerco di portare anche nel mio lavoro tecnico.

La fusione tra il rigore tecnico e la libertà creativa, che caratterizzano i miei studi, rappresenta il mio punto di forza permettendomi di affrontare ogni progetto con curiosità, spirito critico e la voglia di trovare soluzioni innovative.

II. NUCLEAR INSTRUMENTS

II.1 - Introduzione



Nuclear Instruments è un'azienda fondata e gestita da ricercatori, con l'obiettivo di costruire strumenti innovativi a supporto della comunità scientifica mondiale. La sua missione è fornire strumenti affidabili e ad alte prestazioni, essenziali per la ricerca sperimentale in fisica e ingegneria.

L'azienda nel corso degli anni, ha progettato e realizzato una vasta gamma di sistemi elettronici e software personalizzati, utilizzati oggi in alcuni dei più importanti centri di ricerca a livello internazionale.

Operano in ambiti di ricerca che spaziano dall'esplorazione di nuovi materiali allo studio della struttura della materia nei grandi collider, dall'astrofisica all'aerospazio.

Ciò che contraddistingue Nuclear Instruments è la profonda competenza tecnica del team, di cui oltre metà dei membri è in possesso di un dottorato in fisica o ingegneria. Tutti i progetti vengono sviluppati internamente, in un ambiente che garantisce la massima riservatezza e sicurezza dei dati, grazie a infrastrutture private che non si affidano a servizi cloud pubblici.

La mia esperienza in azienda, oggetto di questa tesi, si inserisce proprio in questo contesto di eccellenza tecnologica, permettendomi di contribuire e di imparare da un ambiente altamente stimolante.



II.II - Tecnologie offerte

Nuclear Instruments offre un servizio completo, che copre l'intero ciclo di vita del prodotto, dall'ideazione alla produzione finale. Questo approccio integrato garantisce la massima qualità, affidabilità e coerenza in tutte le fasi dello sviluppo.

Sviluppo Hardware

La progettazione hardware presso Nuclear Instruments si distingue per l'utilizzo di strumenti di simulazione circuitale (ad esempio basati su modelli SPICE) e per la stretta collaborazione tra i reparti E-CAD e M-CAD. Questa integrazione consente di gestire progetti complessi, come rivelatori personalizzati o schede destinate a essere integrate in satelliti e telescopi.

I progetti vengono costantemente monitorati e aggiornati grazie a un sistema interno di gestione, che collega i dati di progettazione con i dati di magazzino e di produzione, riducendo il rischio di errori e garantendo una visione completa e precisa delle risorse impiegate.

Inoltre, l'uso di sistemi di versionamento con Git e di pipeline automatizzate CI/CD (Continuous Integration / Continuous

Delivery) consente di mantenere uno sviluppo ordinato e affidabile, assicurando sempre la tracciabilità e la qualità del progetto.

Sviluppo Software e Firmware

Un aspetto distintivo dell'azienda è la grande attenzione alla stabilità e riproducibilità dell'ambiente di sviluppo software. La maggior parte dei progetti software e firmware viene sviluppata all'interno di container Docker ospitati sui server aziendali. Questo approccio garantisce coerenza tra i diversi ambienti di sviluppo e semplifica l'onboarding dei nuovi membri del team.

Quando necessario, vengono impiegati ambienti virtual machine (VM), per rispondere a requisiti specifici. In ogni caso, il controllo delle versioni tramite Git e l'integrazione con pipeline CI/CD assicura che ogni commit generi test e rilasci automatizzati, migliorando la qualità, la manutenibilità e l'efficienza del software prodotto.

Produzione e Procurement

Nuclear Instruments gestisce con grande attenzione anche la fase di produzione, collaborando esclusivamente con fornitori europei per la realizzazione dei PCB e curando internamente l'assemblaggio, interamente in Italia. Questo approccio

garantisce la conformità agli standard internazionali come RoHS, UL e REACH, e minimizza i rischi legati alla catena di fornitura.

Un sistema avanzato di gestione dei componenti consente di prevedere con precisione i livelli di magazzino, in base ai cicli produttivi pianificati e agli ordini di componenti già confermati. In questo modo, l'azienda mantiene un alto livello di continuità e affidabilità, elementi essenziali nei progetti scientifici e industriali.

Test e Validazione

Tutti i prodotti realizzati vengono sottoposti a rigorosi test di validazione e calibrazione per garantire il massimo livello di qualità e affidabilità. Ogni scheda elettronica viene testata singolarmente prima di essere integrata nei prodotti finali, con la registrazione dei dati e la completa tracciabilità di ogni componente.

Il sistema interno gestisce l'intero processo di testing, memorizzando in modo sicuro i dati e generando report dettagliati o riassunti che possono essere messi a disposizione dei clienti. Questo garantisce trasparenza, ma soprattutto permette di avere la certezza che ogni prodotto rispetti gli elevati standard qualitativi richiesti in ambito scientifico.

II.III - Partnership



Un elemento chiave dell'attività di Nuclear Instruments è la collaborazione con CAEN S.p.A., una sinergia unica nel mondo della fisica nucleare che combina competenze complementari per offrire soluzioni tecnologiche all'avanguardia. Insieme, le due aziende occupano una posizione di leadership mondiale, consolidata grazie a una lunga tradizione di innovazione e al continuo supporto alla comunità scientifica internazionale.

Il catalogo congiunto di Nuclear Instruments e CAEN, che comprende oltre 400 strumenti diversi di misura, offre un'ampia gamma di soluzioni pronte all'uso per la ricerca e lo sviluppo in migliaia di laboratori e progetti sperimentali in tutto il mondo.

Questa collaborazione garantisce strumenti di precisione per applicazioni in fisica fondamentale, spettroscopia nucleare, fisica delle alte energie, medicina nucleare, astrofisica e fusione nucleare.

CAEN S.p.A. è riconosciuta come l'unica azienda al mondo a offrire una gamma completa di sistemi di alimentazione ad alta/bassa tensione e moduli di acquisizione dati interamente conformi agli standard IEEE per la fisica nucleare e delle particelle. I suoi prodotti sono utilizzati da un'ampia gamma di clienti, tra cui ingegneri, scienziati e professionisti del settore tecnico, che si affidano a questi strumenti per raggiungere i propri obiettivi in modo più rapido ed efficace.

Nuclear Instruments si affida a CAEN per la distribuzione dei propri prodotti, creando una sinergia che consente di portare sul mercato soluzioni elettroniche e software integrate, progettate con la massima cura e adatte a soddisfare le esigenze più avanzate della comunità scientifica e industriale.

Grazie alla visione condivisa di eccellenza ed innovazione, le due aziende non solo rispondono alle necessità attuali della ricerca, ma continuano a plasmare il futuro della tecnologia nucleare, sostenendo lo sviluppo di soluzioni sempre più avanzate per affrontare le sfide globali.

II.IV - Ambiente lavorativo

Nuclear Instruments conta attualmente un team di 20 giovani ingegneri, fisici e informatici altamente qualificati.

L'azienda è caratterizzata da una forte apertura verso i giovani e investe nella loro crescita e sviluppo professionale. La stragrande maggioranza delle persone che oggi fanno parte dell'azienda ha iniziato il proprio percorso con uno stage o apprendistato.

Sebbene sono un'azienda, il loro modello operativo si configura come un vero e proprio laboratorio di ricerca, caratterizzato da un forte orientamento all'innovazione e un profilo internazionale, collaborando attivamente con istituzioni scientifiche di rilievo mondiale.

L'ambiente di lavoro si distingue per la flessibilità oraria e la possibilità di affrontare progetti stimolanti.

In questo ambiente, i dipendenti sono incentivati a sperimentare e a proporre soluzioni innovative, sapendo di poter contare su un supporto costante.

III. IL MIO RUOLO NELL'AZIENDA

III.I - Responsabilità e contributi tecnici

Durante il mio tirocinio presso Nuclear Instruments, ho avuto la responsabilità di seguire in prima persona lo sviluppo di applicazioni web avanzate per la gestione e la visualizzazione dei dati generati dagli strumenti di misura dell'azienda.

Responsabilità assegnate

Il mio ruolo prevedeva:

- La progettazione completa dell'applicazione, in stretta collaborazione con i team di ingegneria, per garantire che le interfacce rispondessero pienamente alle esigenze tecniche e operative.
- L'implementazione e ottimizzazione di tutte le funzionalità necessarie, assicurandomi che le interfacce web fossero reattive, di facile utilizzo e ad alte prestazioni.
- L'adozione delle migliori pratiche di sviluppo, con particolare attenzione alla qualità del codice e alla chiarezza del design visivo.

Obiettivi formativi e di orientamento

Queste attività hanno rispettato pienamente gli obiettivi formativi previsti dal mio piano di tirocinio:

- Progettazione dello sviluppo dell'applicazione, comprensiva delle logiche di progettazione software, delle metodologie architetturali e delle tecniche di programmazione, con un'attenzione particolare alla programmazione orientata agli oggetti e ai framework più utilizzati nel settore.
- Definizione del disegno dell'applicazione, inclusa la padronanza di HTML5, CSS e dei linguaggi emergenti, con l'obiettivo di creare un layout leggibile e funzionale, capace di integrare grafici e rappresentazioni visive per un design adeguato all'utente finale.

Contributi tecnici: progettazione e sviluppo completi

Dal punto di vista tecnico, mi sono occupato in autonomia di tutti gli aspetti dello sviluppo dell'applicazione, sia per quanto riguarda il backend sia per la parte frontend.

Per la logica di backend, ho progettato e implementato l'API necessaria, garantendo un'elaborazione dei dati efficiente e scalabile. Per quanto riguarda il frontend invece, ho realizzato l'interfaccia utente, sfruttando framework e stili moderni.

Inoltre, mi sono occupato di curare l'integrazione completa tra il frontend e il backend, assicurandomi che ogni funzionalità fosse perfettamente coerente e che le performance complessive dell'applicazione rispondessero agli standard qualitativi richiesti dall'azienda.

IV. ZEUS

IV.I - Descrizione generale

ZEUS è un software intuitivo, progettato per il controllo e la gestione di alimentatori compatti di alta tensione. Si tratta di uno strumento versatile e adatto a chi opera in contesti sperimentali o produttivi che richiedono precisione e sicurezza.

La piattaforma consente di avere controllo, monitoraggio e logging in un unico ambiente integrato, facilitando la gestione quotidiana e aumentando l'efficienza operativa. Ogni parametro degli alimentatori può essere configurato e controllato tramite interfaccia grafica.

Progettato per adattarsi a diversi scenari, sia di laboratorio sia di produzione, ZEUS offre tracciabilità completa dei dati e garantisce sempre la massima affidabilità. Le funzionalità di logging e monitoraggio consentono di mantenere sotto controllo tutti gli aspetti operativi, assicurando continuità e sicurezza anche in contesti critici.

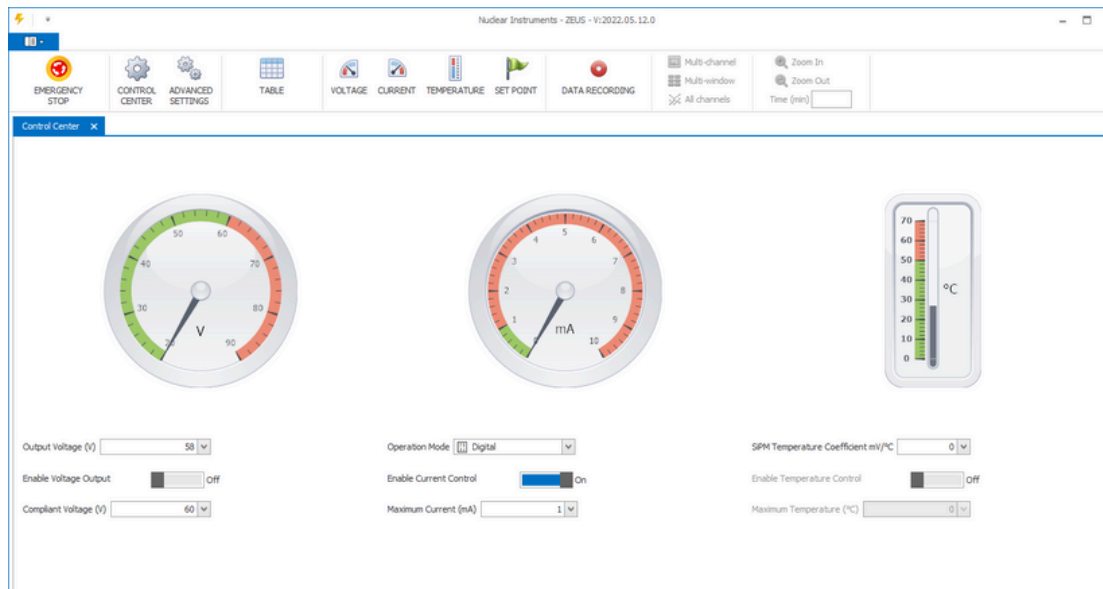
IV.II - Il primo Zeus

La prima versione del software ZEUS sviluppata da Nuclear Instruments rappresentava una soluzione efficace e funzionale per il controllo e la gestione degli alimentatori ad alta tensione. Tuttavia, presentava alcune limitazioni e vincoli legati all'ambiente operativo.

In particolare, il vecchio ZEUS poteva essere eseguito esclusivamente su sistemi operativi Windows, limitando di fatto la portabilità e la possibilità di integrazione in contesti più eterogenei o con esigenze di piattaforme differenti.

Dalle immagini dell'applicativo, si possono osservare i diversi pannelli e funzionalità del software:

Pannello principale con indicatori analogici per output voltage, operation status e temperature, insieme a caselle di input per regolare i parametri principali.



Grafici in tempo reale che mostrano l'andamento di parametri come la tensione e la corrente, offrendo una visualizzazione chiara e immediata.

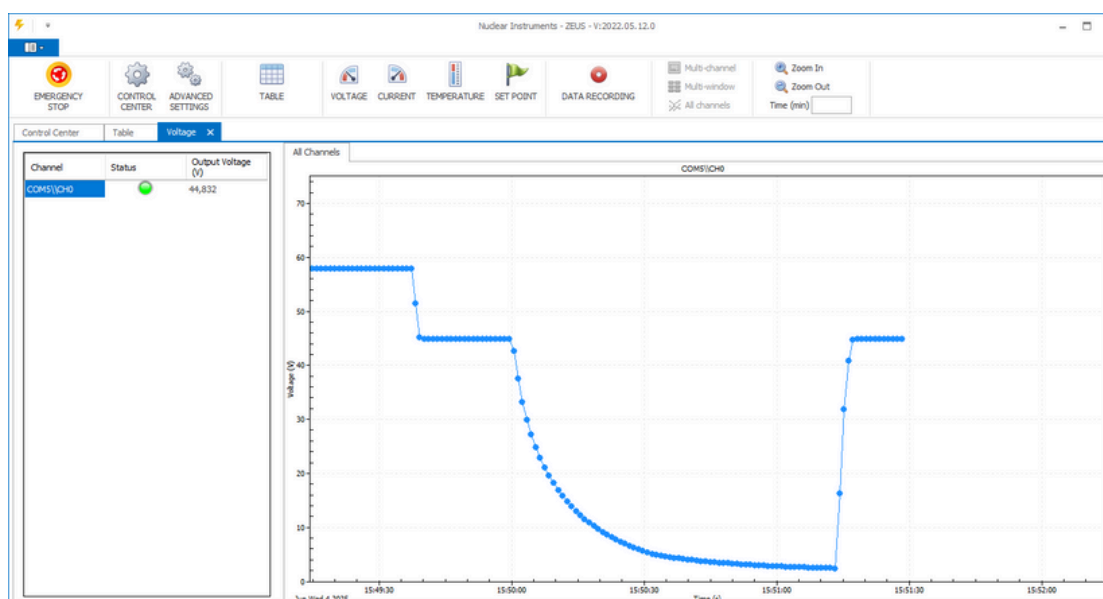
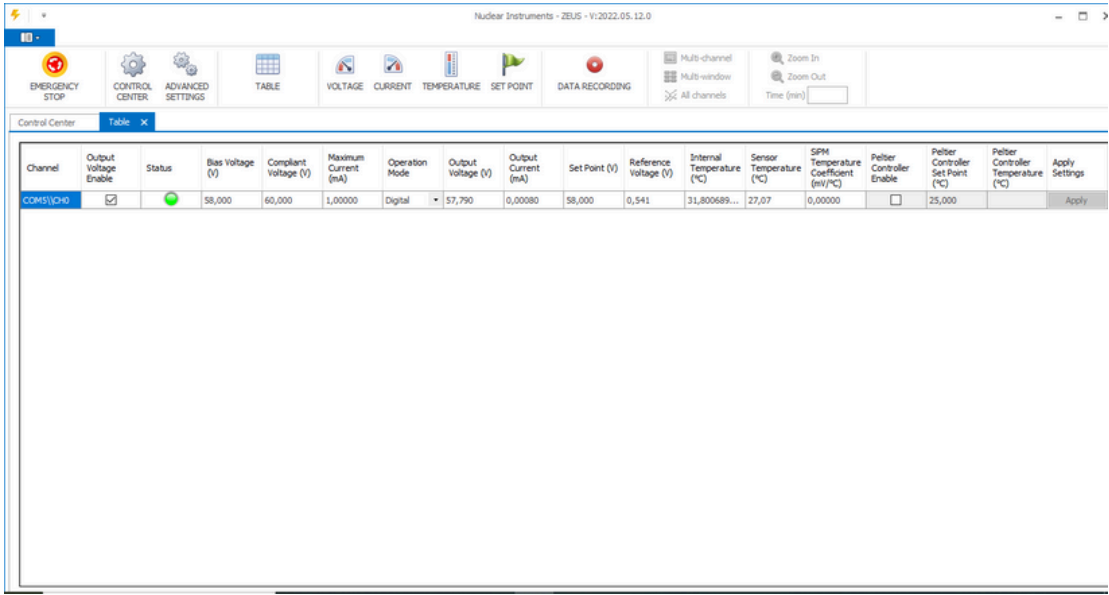


Tabelle di controllo dettagliate per ogni canale, che riportano i parametri fondamentali e consentono una gestione precisa del sistema.



The screenshot shows the 'Control Center' window of the 'Nuclear Instruments - ZEUS' software. The 'Table' tab is selected, displaying a detailed table of parameters for the 'COM1\CH0' channel. The table includes columns for various electrical and thermal parameters, with some cells containing checkboxes and status indicators.

Channel	Output Voltage Enable	Status	Bias Voltage (V)	Compliant Voltage (V)	Maximum Current (mA)	Operation Mode	Output Voltage (V)	Output Current (mA)	Set Point (V)	Reference Voltage (V)	Internal Temperature (°C)	Sensor Temperature (°C)	SPM Temperature Coefficient (mV/°C)	Peltier Controller Enable	Peltier Controller Set Point (°C)	Peltier Controller Temperature (°C)	Apply Settings
COM1\CH0	☒	●	58,000	60,000	1,00000	Digital	57,790	0,00080	58,000	0,541	31,800689...	27,07	0,00000	☐	25,000		Apply

L'interfaccia è caratterizzata da uno stile classico e lineare, con icone in alto per accedere alle varie funzionalità (ad esempio: Emergency Stop, Control Center, Advanced Settings, Data Recording) e una struttura a schede per facilitare la navigazione tra le diverse sezioni.

Limitazioni e motivazioni per l'evoluzione

Nonostante la solidità e l'affidabilità dimostrate negli anni, il vincolo di compatibilità esclusiva con Windows ha

rappresentato un limite significativo per l'adozione del software in contesti che richiedevano flessibilità multiplatforma e personalizzazione dell'interfaccia.

L'evoluzione del software ZEUS, di cui parlerò più avanti, ha risposto proprio a queste necessità, rendendolo più versatile e allineato alle esigenze contemporanee.

IV.III - Requisiti principali

Lo sviluppo della nuova versione del software ZEUS è partito da una serie di requisiti fondamentali espressi dall'azienda.

Scalabilità e compatibilità multi-piattaforma

Uno degli aspetti principali richiesti era la possibilità di eseguire il software su qualsiasi sistema operativo, eliminando la limitazione storica alla sola piattaforma Windows.

Supporto a connessioni multiple

Per adattarsi alle esigenze di diversi scenari sperimentali, ZEUS doveva essere in grado di gestire:

- Connessioni seriali.
- Connessioni Ethernet (non ancora esistenti).
- Connessioni I²C.

Questa versatilità nelle connessioni ha permesso di controllare e monitorare una vasta gamma di strumentazioni e sistemi, poichè la prima versione è stata sviluppata per una particolare famiglia di alimentatori.

Personalizzazione della tabella di controllo

Un altro requisito fondamentale era la possibilità di personalizzare la tabella di controllo. Gli utenti dovevano poter scegliere quali parametri visualizzare e adattare l'interfaccia alle specifiche modalità di lavoro.

Interfaccia moderna e intuitiva

La necessità di una interfaccia utente moderna, chiara e reattiva era centrale. Lo scopo era rendere più intuitiva l'interazione con il software, facilitando l'accesso a tutte le funzionalità e garantendo un'esperienza d'uso più immediata e piacevole.

Raggruppamento dei canali

Il raggruppamento dei canali è stato individuato come un elemento chiave per migliorare la gestione e il monitoraggio degli alimentatori.

Questa funzionalità doveva consentire di creare gruppi logici di canali e applicare comandi e configurazioni simultanee su più canali.

Miglioramento dei grafici

Infine, un requisito essenziale, era rappresentato dal miglioramento dei grafici, per poter ottenere visualizzazioni più leggibili e chiare. Era importante non solo aumentare la precisione e la qualità complessiva delle rappresentazioni grafiche, ma anche consentire una personalizzazione nella visualizzazione dei vari canali, in modo da adattarsi meglio alle diverse esigenze.

Questi requisiti principali hanno rappresentato la base su cui ho impostato le attività di sviluppo durante il mio tirocinio, con l'obiettivo di fornire un software più flessibile, intuitivo e capace di adattarsi.

IV.IV - Sviluppo di Zeus

Lo sviluppo della nuova versione di ZEUS ha rappresentato un momento fondamentale della mia esperienza in azienda e ha richiesto un'attenzione particolare all'adozione delle tecnologie più adatte, alla definizione di un ambiente di sviluppo moderno e alla realizzazione di un software modulare e flessibile.

IV.IV.I - Ambiente di sviluppo

Per garantire coerenza e stabilità in tutte le fasi di sviluppo, ho scelto di adottare ambienti moderni e versatili, capaci di rispondere alle esigenze di progetti complessi e multi-piattaforma.

Questa scelta si è rivelata vincente, poiché ha semplificato notevolmente il processo di sviluppo, riducendo le complessità e i possibili punti critici. Inoltre, grazie alla facilità di integrazione di estensioni, ho potuto organizzare in modo più efficiente tutte le attività del progetto.

IV.IV.I.I – Visual Studio Code

L'ambiente principale che ho scelto per lo sviluppo di ZEUS è stato Visual Studio Code.

La sua interfaccia leggera e flessibile si era perfetta per gestire un progetto così articolato, che includeva molteplici linguaggi e framework. Visual Studio Code mi ha inoltre permesso di personalizzare l'ambiente di sviluppo in modo rapido, grazie al suo ampio ecosistema di estensioni. Ho potuto così aggiungere facilmente funzionalità specifiche e strumenti legati al progetto, mantenendo sempre un flusso di lavoro fluido e organizzato.

Ho apprezzato la sua integrazione nativa con i sistemi di controllo delle versioni, come Git, e la possibilità di lavorare direttamente con container Docker. Questo ha reso lo sviluppo ancora più efficiente e ordinato, consentendomi di mantenere sempre alta la qualità del codice e del processo stesso.



IV.IV.I.I.I – Strumenti di supporto

Per arricchire ulteriormente l'ambiente di sviluppo e rendere più semplici le operazioni quotidiane, ho fatto ricorso a una serie di strumenti di supporto molto utili. Ad esempio, ho integrato plugin e tool dedicati al debugging, che mi hanno permesso di identificare e risolvere in modo rapido eventuali problemi, senza interrompere il flusso di lavoro. Infine, ho fatto ampio uso di strumenti per il refactoring e la formattazione automatica del codice, grazie ai quali sono riuscito a mantenere il codice sempre ordinato, facilitando così la manutenzione a lungo termine.

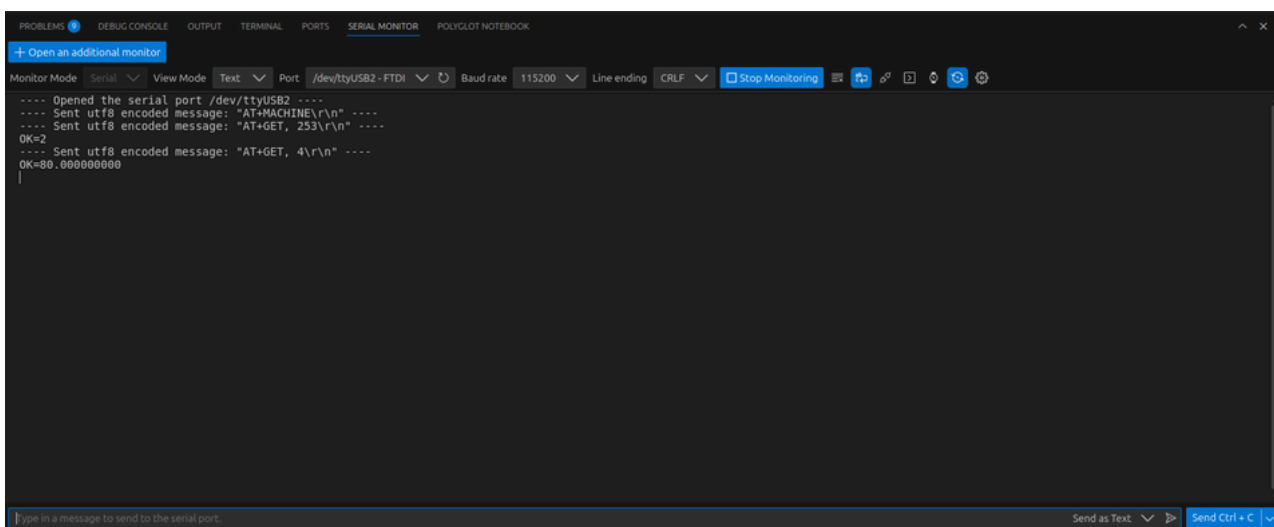
IV.IV.I.I.I.I – Serial Monitor

Un componente fondamentale tra gli strumenti di supporto che ho utilizzato è stato il Serial Monitor, un'interfaccia diretta e intuitiva per comunicare con i dispositivi seriali.

Grazie al Serial Monitor, ho potuto inviare comandi ai dispositivi collegati e ricevere in tempo reale le risposte inviate dall'hardware, visualizzando immediatamente i dati e le eventuali anomalie.

Questa interfaccia si è rivelata particolarmente utile nelle fasi di sviluppo, consentendomi di monitorare e diagnosticare facilmente comportamenti inattesi.

L'utilizzo del Serial Monitor ha permesso di verificare la correttezza delle sequenze di comando e di individuare facilmente, qualora ne fosse necessario, se gli errori riscontrati fossero dovuti a malfunzionamenti degli alimentatori o bug del software.



IV.IV.II - Tecnologie utilizzate

Lo sviluppo di ZEUS ha richiesto l'adozione di tecnologie moderne e performanti, capaci di garantire un'esperienza utente fluida e una gestione efficiente delle comunicazioni hardware. Di seguito, una panoramica dei principali strumenti e framework utilizzati: Wails, Go, React e Lightning Chart.

IV.IV.II.I – Wails

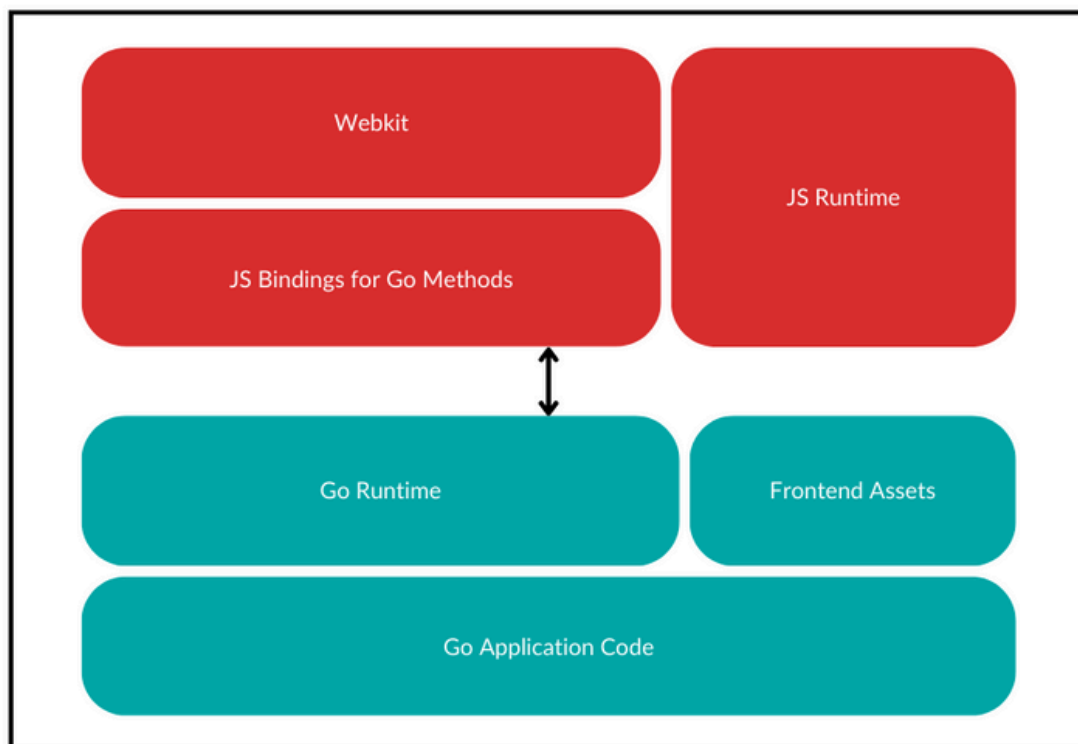
Per la realizzazione dell'applicazione desktop ZEUS, ho deciso di adottare Wails, seguendo il consiglio dell'azienda. Si tratta di un framework moderno e leggero che consente di sviluppare applicazioni desktop native combinando Go per la logica di backend con le più moderne tecnologie web per la parte di interfaccia utente.

Wails si è rivelato un'alternativa a Electron, in quanto è stato progettato specificamente per integrare la potenza e la flessibilità di Go con la ricchezza dei nuovi framework frontend web moderni come React, Svelte, Vue, Preact, Lit e anche Vanilla JS, sia in JavaScript che in TypeScript.

Inoltre, Wails offre piena compatibilità con Windows, macOS e Linux, permettendo quindi di realizzare un'applicazione multiplatforma partendo da un'unica base di codice.

Grazie a queste caratteristiche, è stato possibile creare un'applicazione desktop stabile e reattiva su qualsiasi sistema operativo.

Components of a Wails App



IV.IV.II.I.I – Go

Sviluppato da Google, Go (noto anche come Golang) si distingue per la sua efficienza e semplicità. Grazie a queste qualità Go è stato fondamentale per la realizzazione di ZEUS.

In particolare, Go ha offerto prestazioni elevate, grazie alla velocità e a un basso consumo di risorse. Questo si è tradotto in tempi di risposta rapidi e in una stabilità ottimale anche quando l'applicazione si trovava a gestire operazioni complesse.

Un altro aspetto fondamentale è la gestione integrata della concorrenza. Go mette a disposizione le cosiddette goroutine, che consentono di gestire più operazioni simultaneamente in modo semplice ed efficiente. Questa caratteristica si è rivelata cruciale per ZEUS, per poter dialogare con i dispositivi in tempo reale senza compromettere la stabilità del sistema.



IV.IV.II.I.II – React

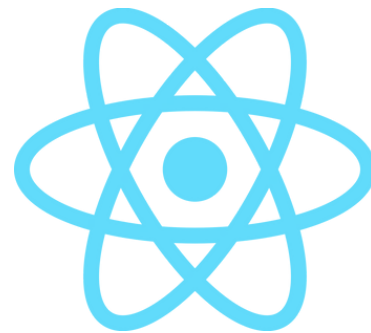
Per la realizzazione del frontend di ZEUS, ho scelto di utilizzare React, una dei framework più apprezzati e diffusi per la creazione di interfacce web moderne.

Uno dei principali punti di forza di React è la sua struttura a componenti, che consente di suddividere l'interfaccia in elementi riutilizzabili. Ciò ha permesso di migliorare la scalabilità del progetto e mantenere il codice sempre ordinato e coerente.

Nello sviluppo di ZEUS, React si è dimostrato prezioso per creare un'interfaccia utente intuitiva e moderna, che risponde ai requisiti di chiarezza e semplicità d'uso richiesti dal progetto.

Ho potuto gestire in modo dinamico i dati provenienti dagli alimentatori, visualizzandoli in tempo reale e integrato librerie specializzate, come LightningChart per la rappresentazione grafica dei dati.

Grazie a React, l'interfaccia di ZEUS si è rivelata solida e flessibile, capace di adattarsi rapidamente ai cambiamenti nei dati e nelle configurazioni, e di offrire una base robusta per le future evoluzioni del software.



IV.IV.II.I.II.I – LightningChart

Per la visualizzazione e l'analisi in tempo reale dei dati generati dagli alimentatori, ho scelto di utilizzare la libreria LightningChart, uno strumento professionale ad alte prestazioni.

Una delle qualità principali di LightningChart è quella del rendering performante, infatti riesce a gestire grandi quantità di dati garantendo aggiornamenti fluidi e precisi.

La sua ricca gamma di grafici, ha permesso di rappresentare in modo chiaro le misure acquisite dagli alimentatori.

Un aspetto importante da sottolineare è che LightningChart è una libreria a pagamento, la cui licenza necessaria per l'utilizzo è stata fornita direttamente dall'azienda, che già da diversi anni fa uso della versione 5.2.1. Questo investimento ha permesso di integrare nel software grafici interattivi, in grado di visualizzare in tempo reale i parametri più significativi.



IV.IV.III – Sistema modulare

ZEUS è stato progettato fin dall'inizio con un'architettura modulare, in cui ogni componente del sistema è stato incapsulato in moduli indipendenti. Questo approccio ha semplificato notevolmente la manutenzione e l'evoluzione del software, rendendo più agevole l'introduzione di nuove funzionalità o il supporto a nuovi dispositivi, senza influenzare negativamente le altre parti del sistema.

Ogni modulo è dedicato a gestire una parte specifica del sistema. Questo ha migliorato la leggibilità del codice, facilitando il lavoro di sviluppo e di manutenzione.

Un altro aspetto fondamentale è la netta separazione tra la logica di backend, gestita in Go, e il frontend realizzato con React. Ognuna di queste due parti ha un proprio ciclo di vita, consentendo di sviluppare e mantenere le due parti parallele e indipendenti.

Zeus è stato progettato per essere predisposto all'integrazione di nuovi dispositivi. Il sistema è stato concepito fin dall'inizio con un'architettura flessibile ed estendibile, che consente di aggiungere facilmente moduli per il supporto a nuovi alimentatori.

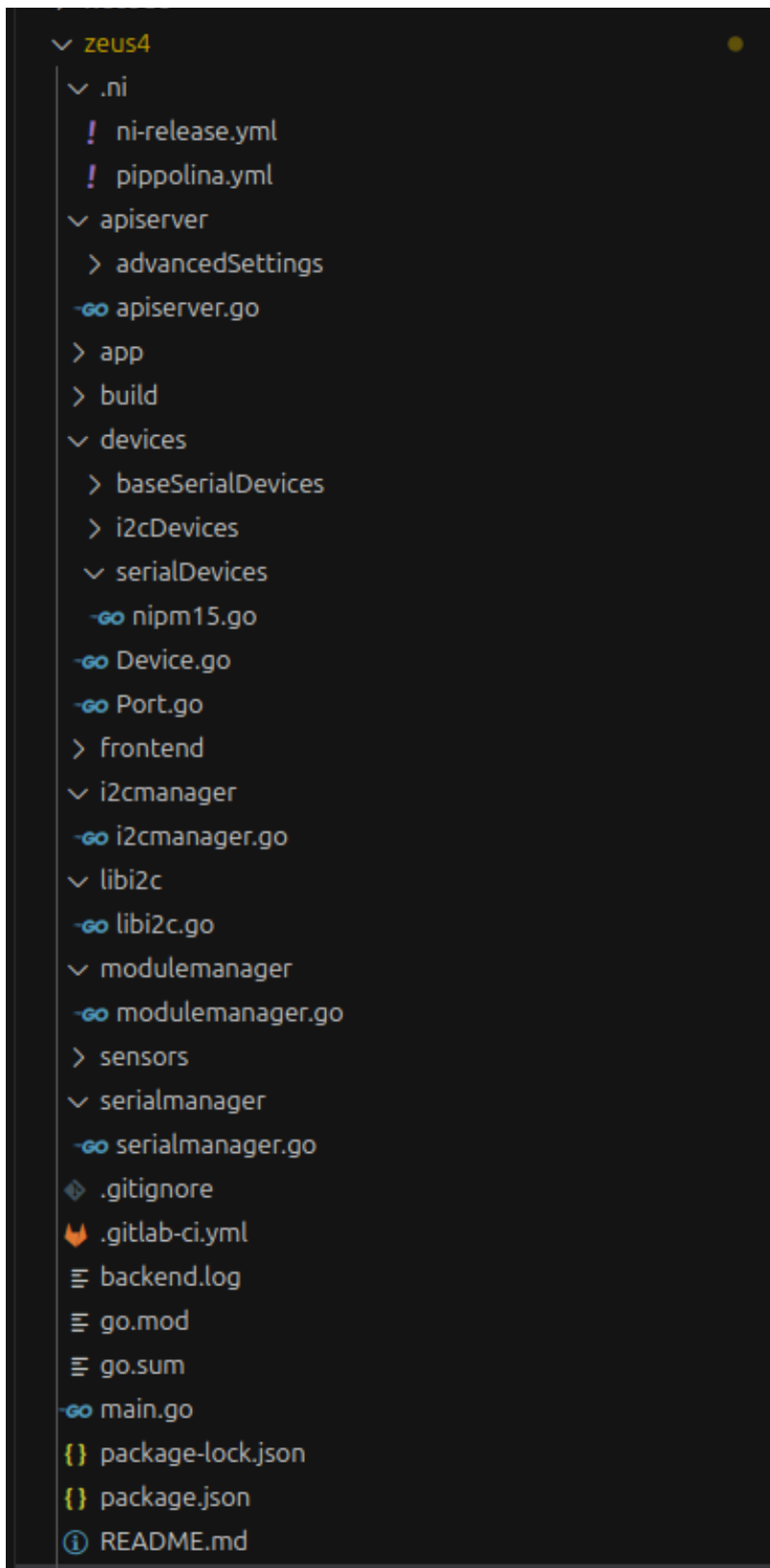
IV.IV.IV – Architettura del Backend

Organizzazione e moduli principali

Il codice sorgente del backend è organizzato in moduli ben definiti, ciascuno dei quali gestisce uno specifico ambito:

- `apiserver`: contiene il server HTTP responsabile di gestire le richieste provenienti dal frontend e di instradarle verso i moduli competenti.
- `devices`: modulo dedicato alla gestione dei dispositivi, suddiviso in:
 - `baseSerialDevices` e `serialDevices`: definiscono le strutture e le operazioni comuni per la comunicazione con i dispositivi seriali.
 - `i2cdevices`: definisce la struttura e le operazioni comuni per la comunicazione con i dispositivi i2c
 - `RegisterDevice.go`, che contiene i registri per la gestione delle periferiche;

- Nipm15.go e A7585d: descrivono rispettivamente il le specifiche del dispositivo e le informazioni specifiche.
- Device.go: continene la struct principale dei devices.
- serialmanager e i2cmanager: moduli che gestiscono la comunicazione seriale e i2c, dall'apertura delle porte all'invio e ricezione dei comandi.
- modulemanager: modulo ausiliare che conferma la partenza dell'API.
- libi2c: contiene la libreria principale con cui avviene la comunicazione con l'i2c, scritta internamente dall'azienda.
- .ni: continene rispettivamnete i file per configurare e automatizzare la release di zeus.
- advancedsettings: contiene i file json degli advanced settings, poichè ogni dispositivo ha i propri e si possono caricare dall'esterno.



Gestione delle comunicazioni

La comunicazione con i dispositivi hardware è gestita in modo flessibile.

Funzioni come `GetProductCode` e `ReadDeviceInfo` estraggono il valore del proprio registro di riferimento e lo inviano, aspettando poi una risposta.

```
func ReadDeviceInfo(device devices.BaseDevice) ([]byte, error) {  
    info := DeviceInfo{}  
  
    // Lettura valori dai registri  
  
    info.HvEnable = GetHvEnable(device)  
    info.ProductCode = GetProductCode(device)  
    info.FWVersion = GetFWVersion(device)  
    info.HWVersion = GetHWVersion(device)  
    info.SerialNumber = GetSerialNumber(device)  
    info.SensorTemperature = GetTemperature(device)  
    info.OutputVoltage = GetOutVoltage(device)  
    info.VoltageSetPoint = GetVoltageSetPoint(device)  
    info.Mode = GetMode(device)  
    info.LUTEnable = GetLUTEnable(device)  
    info.I2CBaseAddress = GetI2CBaseAddress(device)
```

```
func GetProductCode(device devices.BaseDevice) string {  
    // Legge il valore del registro del codice prodotto  
    productCodeResponse, err := Read(device, device.RegisterDevice.RegProductCode)  
    if err != nil {  
        log.Println("Errore nella lettura del codice prodotto:", err)  
        return "" // Restituisce stringa vuota in caso di errore  
    }  
  
    //Pulisce la risposta ricevuta  
    productCode := cleanSerialResponse(productCodeResponse) // Rimuove caratteri extra  
  
    return productCode  
}
```

ReadSerial e SendSerialCommand sono funzioni centrali che costruiscono e inviano comandi ai dispositivi tramite la porta seriale, assicurandosi che la sintassi sia corretta (ad esempio, terminando i comandi con `\r\n`) e gestendo eventuali errori di comunicazione.

```
func Read(device devices.BaseDevice, numberRegister int) (string, error) {
    port, err := devices.GetPort(device)
    if err != nil {
        log.Printf("⚠ Errore nel recupero della porta %s: %v", device.PortName, err)
        return "", err
    }

    switch device.DeviceType {
    case "Serial":
        serialPort, ok := port.(*devices.SerialPort)
        if !ok {
            return "", fmt.Errorf("❌ Il dispositivo %s non è una porta seriale valida", device.PortName)
        }

        // ✅ Possiamo `serialPort.Port` (che è serial.Port) a `ReadSerial`
        return ReadSerial(serialPort.Port, numberRegister), nil
    }
}
```

```
func ReadSerial(port *serial.Port, numberRegister int) string {
    ✨ command := fmt.Sprintf("AT+GET,%d\r\n", numberRegister) // Costruisce il comando
    return SendSerialCommand(port, command)                  // Invia il comando e ritorna la risposta
}
```

Strutture dati

BaseDevice, rappresenta un dispositivo generico, contenendo informazioni come serial number, modello, firmware e hardware.

```
type BaseDevice struct {  
    RegisterDevice *baseSerialDevices.RegisterDevice  
    DeviceType     string  
    Type           string  
    Model          string  
    SerialNumber   string  
    Firmware       string  
    Hardware       string  
    PortName       string  
    ConnectionStatus string  
    Channels        map[string]interface{} // Può contenere canali di tipo diverso a seconda del dispositivo  
}
```

Channel, rappresenta i canali dei dispositivi, con parametri operativi come tensione, corrente, temperatura e molti altri.

```
type Nipm15 struct {  
    devices.BaseDevice  
}  
  
// Struttura Channel per specifiche di ciascun canale  
type Channel struct {  
    Id                string  
    ChannelName       string  
    HvEnable          bool  
    Mode              int  
    VoltageSetPoint   float64  
    CurrentSetPoint   float64  
    RampSpeed         float64  
    MaxVoltage        float64  
    MaxCurrent        float64  
    CTempM2           float64  
    CTempM            float64  
    CTempQ            float64  
    AlfaVout          float64  
    AlfaIout          float64  
    AlfaVref          float64  
    AlfaTref          float64  
    TCoef             float64  
    LUTEnable         bool  
    EnablePI          bool  
    EmergencyStop     bool  
    IZero             bool  
    LUTAddress        int  
}
```

RegisterDevice, mappa i registri hardware, definendo le chiavi e gli offset da utilizzare per leggere o scrivere i dati.

```
// Interfaccia comune per tutti i dispositivi seriali
type RegisterDevice struct {
    RegProductCode  int
    RegFirmware     int
    RegHardware     int
    RegSerialNumber int
    // Controllo globale
    RegHVEnable     int
    RegMode         int
    RegEmergencyStop int

    // Lookup Table e controllo
    RegLUTEnable     int
    RegLUTAddress    int
    RegLUTProgramTemperature int
    RegLUTProgramOutputValue int
    RegLUTLength     int

    // Stato e configurazione
    RegI2CBaseAddress int
    RegStoreOnFlash   int
    RegPinStatus      int

    // Registri dei canali
    RegVoltageSetPoint int
    RegCurrentSetPoint int
    RegRampSpeed       int
    RegMaxVoltage      int
    RegMaxCurrent      int
}
```

Queste strutture sono pensate per essere estensibili e per consentire in futuro l'aggiunta di nuovi tipi di dispositivi o funzionalità.

IV.IV.V – Architettura del frontend

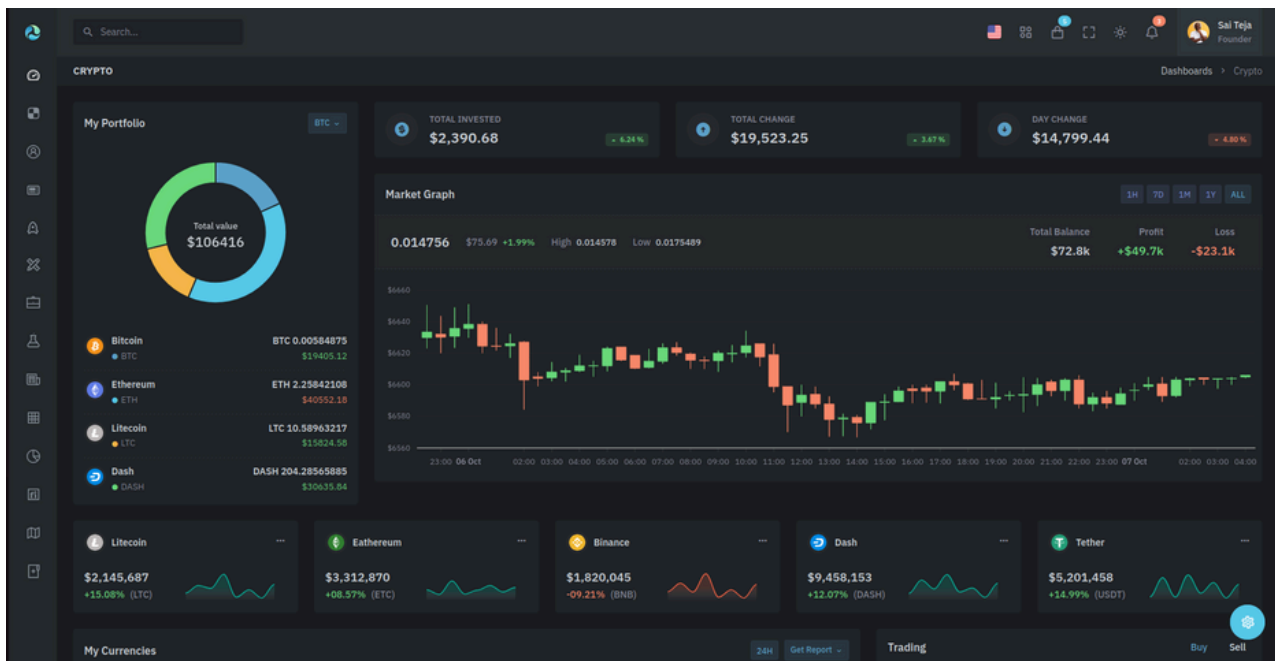
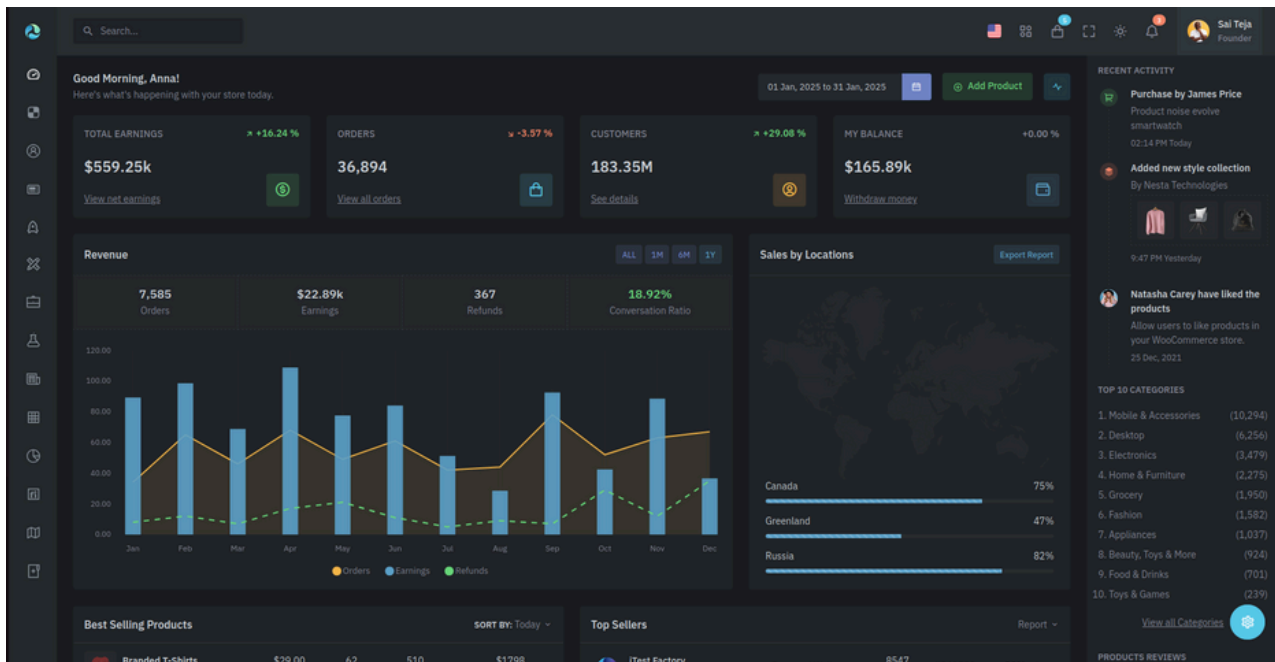
Il frontend di ZEUS è stato sviluppato con uno dei massimi esponenti dei framework per interfacce web: React.

Per accelerare lo sviluppo e garantire un'interfaccia utente accattivante e professionale, ho utilizzato come base un template commerciale, il Velzone Themeforest – Modern, integrando il progetto ZEUS all'interno delle sue logiche preesistenti.

Utilizzo del template Velzone

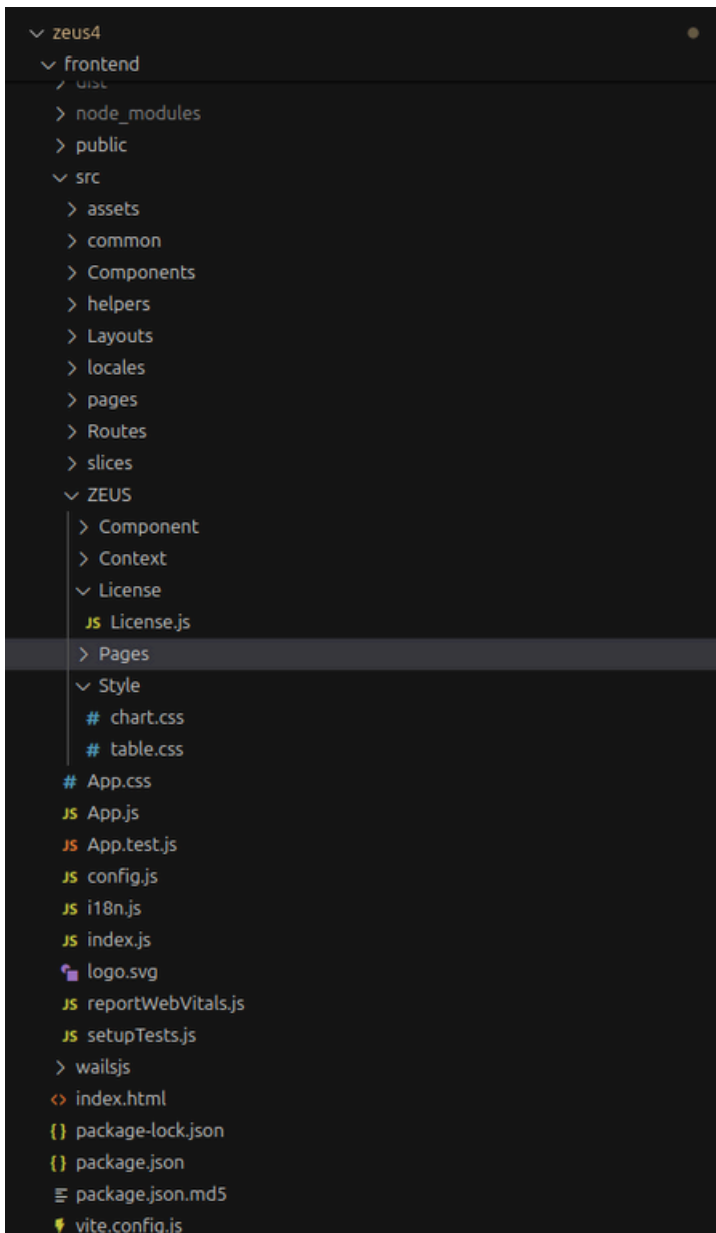
Il template Velzone offre una struttura già organizzata, che include componenti riutilizzabili per la gestione delle interfacce e degli elementi grafici.

Ho adattato il codice dell'applicativo ZEUS a questa architettura, rispettando la filosofia e la gerarchia del template.

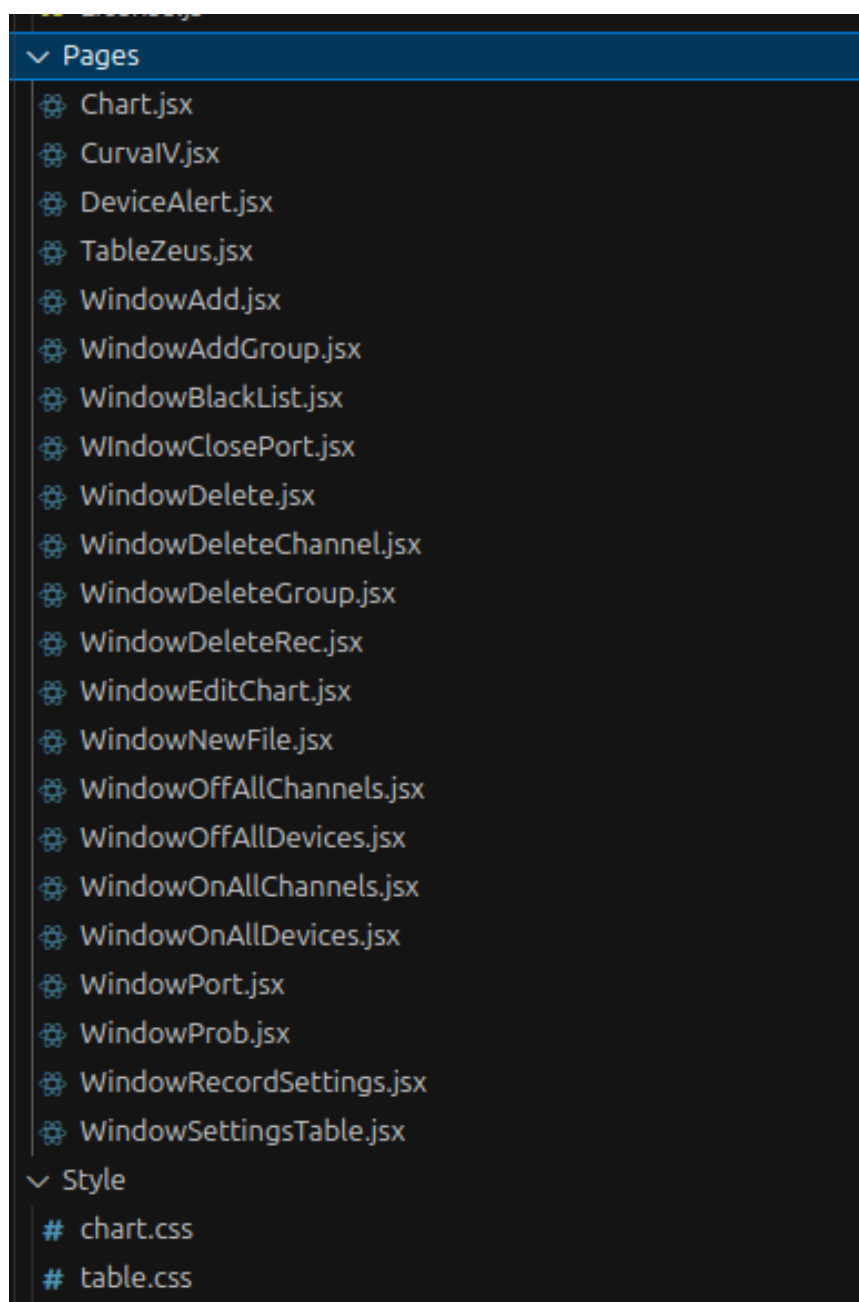


Struttura a componenti

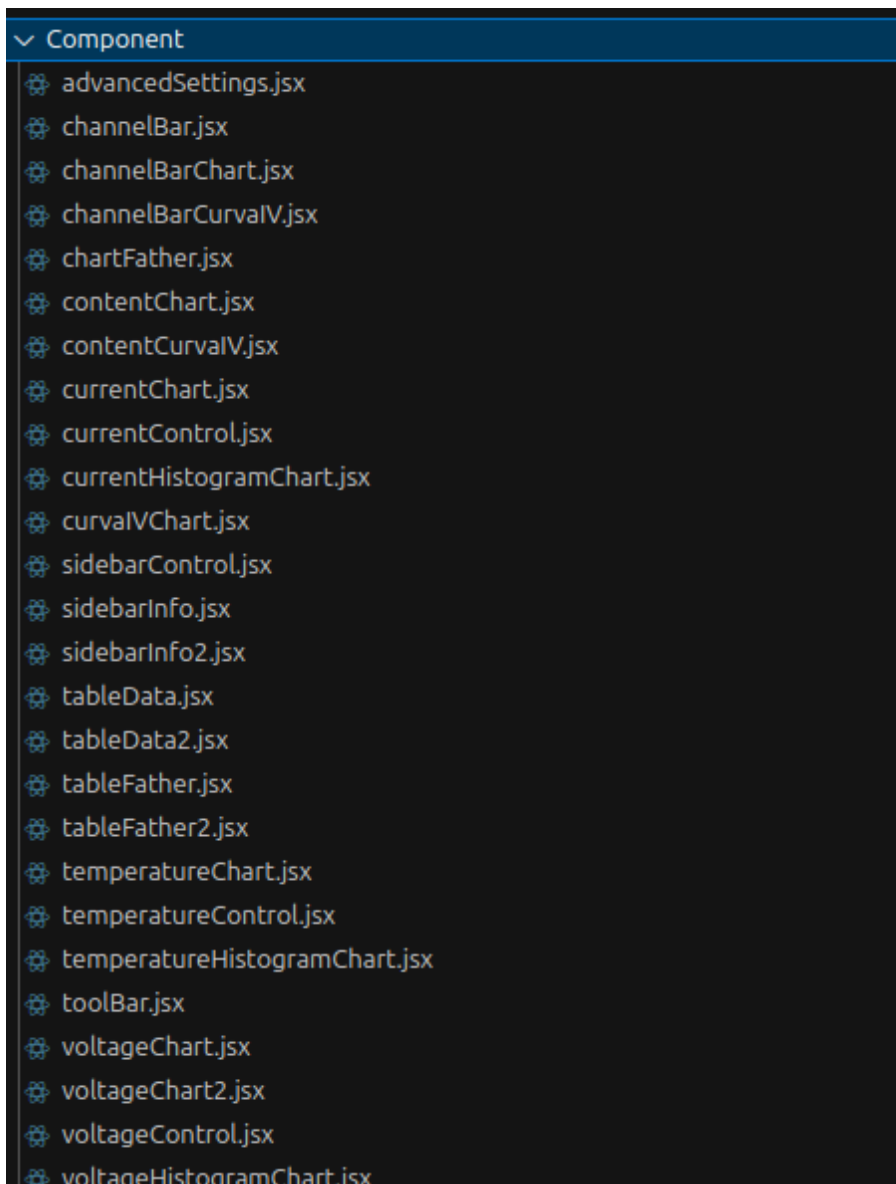
La struttura a componenti è visibile nella suddivisione in cartelle e file, come mostrato nelle immagini.



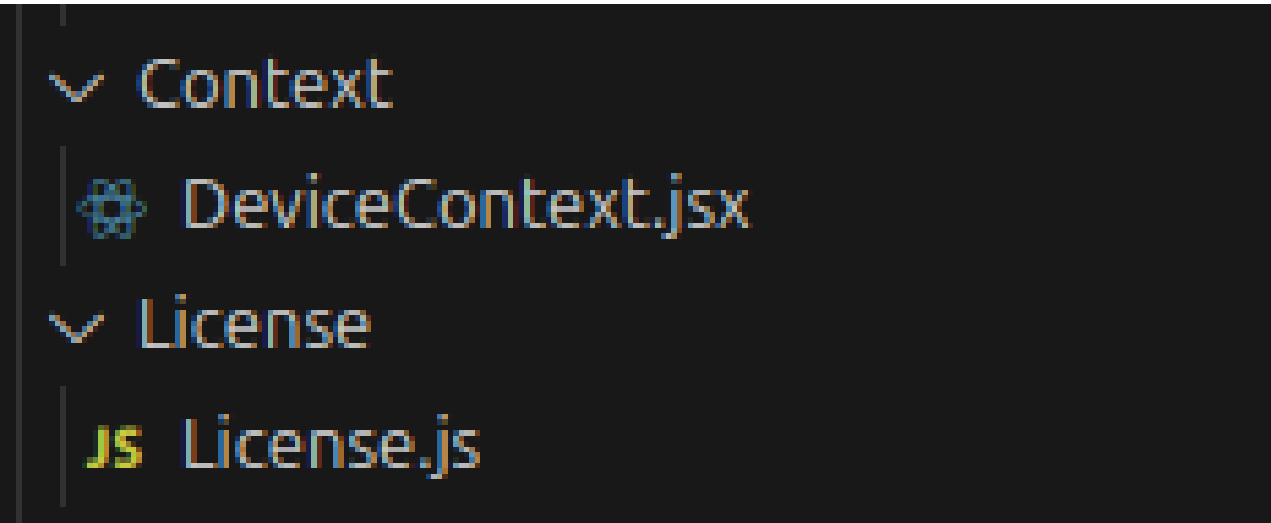
La cartella Pages contiene tutte le pagine principali dell'applicazione (ad esempio Chart, CurvaIV, DeviceAlert, TableZeus), ognuna realizzata come componente autonomo



La cartella Component ospita componenti riutilizzabili e funzioni condivise.



Le cartelle Context e License contengono rispettivamente il file del Context che avvolge l'intero frontend e la licenza LigthninhChart per poter utilizzare i suoi grafici.



```

  Context
    DeviceContext.jsx
  License
    License.js

```

Gestione dello stato e del contesto

Per la gestione e la condivisione dei dati tra i componenti, ho utilizzato la Context API di React.

Il file DeviceContext.jsx crea un contesto globale per i dispositivi, che avvolge l'intera applicazione. L'albero dei componenti è racchiuso nel DeviceProvider, consentendo di gestire facilmente le modifiche di stato all'interno di tutto il progetto.

```
root.render(  
  <DeviceProvider>  
    <Provider store={store}>  
      <React.Fragment>  
        <BrowserRouter basename={process.env.PUBLIC_URL}>  
          <App />  
        </BrowserRouter>  
      </React.Fragment>  
    </Provider>  
  </DeviceProvider>  
)
```

Adattamenti e personalizzazioni

Pur partendo da un template già strutturato, ho realizzato tutte le logiche e le funzionalità specifiche di ZEUS, creando pagine personalizzate per la gestione dei dispositivi, integrando la comunicazione con il backend e coordinando l'interfaccia utente con le API e i dati reali.

La scelta di partire da un template e di integrare ZEUS al suo interno ha permesso di ottenere un'interfaccia moderna, in linea con le richieste dell'azienda.

IV.IV.VI – Comunicazione tra backend e frontend

La comunicazione tra il backend (scritto in Go) e il frontend (basato su React) di ZEUS è stata progettata per garantire stabilità, reattività e facilità di manutenzione.

API RESTful

Il backend espone una serie di API RESTful accessibili tramite il server locale in ascolto sulla porta 8080 (<http://localhost:8080>).

Le principali route gestite includono:

- `/api/devices/`: per la gestione dei dispositivi.
- `/api/record/`: per la gestione e l'archiviazione dei dati.
- `/api/ports/`: per la gestione delle porte seriali ed Ethernet.
- `/api/devices/scan`, `/api/devices/{type}`: per la scansione e l'ottenimento dei dispositivi disponibili.
- Altre route specifiche per la configurazione avanzata e la visualizzazione dei dati.

```
// Avvia il server API
func StartServer(_moduleManager *modulemanager.ModuleManager) {
    log.Println("StartServer è stato chiamato") // Log di debug

    moduleManager = _moduleManager
    mux := http.NewServeMux()

    mux.HandleFunc("/status", corsMiddleware(statusHandler))
    mux.HandleFunc("/api/devices/", corsMiddleware(deviceHandler))
    mux.HandleFunc("/api/record/data", corsMiddleware(recordDataHandler))
    mux.HandleFunc("/api/record/filesize", corsMiddleware(recordFileSizeHandler))
    mux.HandleFunc("/api/record/download", corsMiddleware(recordFileDownloadHandler))
    mux.HandleFunc("/api/record/clear", corsMiddleware(clearRecordFileHandler))
    mux.HandleFunc("/api/device/close", corsMiddleware(closeDeviceHandler))
    mux.HandleFunc("/api/ports/usb", corsMiddleware(listUsbSerialPortsHandler))
    mux.HandleFunc("/api/ports/blacklist", corsMiddleware(setBlacklistedPortsHandler))
    mux.HandleFunc("/api/devices/scan", corsMiddleware(scanDevicesHandler))
    mux.HandleFunc("/api/devices/port", corsMiddleware(getDevicePortHandler))
    mux.HandleFunc("/api/device/reconnect", corsMiddleware(reconnectDeviceHandler))
    mux.HandleFunc("/api/reconnect-port", corsMiddleware(reconnectByPortHandler))
    mux.HandleFunc("/api/record/download/histogram", corsMiddleware(downloadsHistogramHandler))
    mux.HandleFunc("/api/devices/sensor", corsMiddleware(ListAvailableSensors))
    mux.HandleFunc("/api/devices/advanced-settings/", corsMiddleware(getAdvancedSettingsHandler))
    log.Println("Sistema operativo:", runtime.GOOS)

    log.Println("Server API in ascolto su http://localhost:8080")
    err := http.ListenAndServe("0.0.0.0:8080", mux)
    if err != nil {
        log.Fatal("Errore nel server HTTP:", err)
    }
}
```

Il backend utilizza un router multiplexer (`http.NewServeMux()`) che collega ogni endpoint alle rispettive funzioni handler, garantendo modularità e ordine.

Esempio di interazione

Nel frontend, la comunicazione con il backend avviene attraverso le fetch API di JavaScript, che permettono di inviare e ricevere dati in modo semplice ed efficiente.

```
const handleScanning = () => {
  setScanning(true);
  fetch("http://localhost:8080/api/devices/scan", {
    method: "POST"
  })
    .then((response) => {
      if (!response.ok) {
        throw new Error("Errore nella risposta del server");
      }
      return response.text();
    })
    .then((msg) => {
      console.log("Messaggio ricevuto dal server:", msg);
      giveMyDevice();
    })
    .catch((err) => {
      console.error("Errore nella scansione:", err);
    })
    .finally(() => {
      setScanning(false);
    });
};

const giveMyDevice = () => {
  fetch(`http://localhost:8080/api/devices/${selectedType}`)
    .then((response) => response.json())
    .then((data) => {
      setDevicesAddList(Array.isArray(data) ? data : []);
      setSelectedDevice(Array.isArray(data) && data.length > 0 ? data[0] : null);
      setRefresh(false)
    })
    .finally(() => {
      setScanning(false); // ripristina il bottone
    })
    .catch((error) => {
      console.error("Errore nel recupero dispositivi:", error);
      setDevicesAddList([]);
      setSelectedDevice(null);
    });
};
```

La funzione `handleScanning` invia una richiesta POST all'endpoint `/api/devices/scan`, avviando così la scansione dei dispositivi disponibili.

Una volta completata questa operazione, la funzione `giveMyDevice` effettua una richiesta GET verso l'endpoint `/api/devices/${selectedType}`, recuperando le informazioni sui dispositivi rilevati e aggiornando dinamicamente l'interfaccia utente.

In questo modo, i dati elaborati sul server vengono integrati in tempo reale nella vista dell'utente.

IV.IV.VII – Sezioni di ZEUS

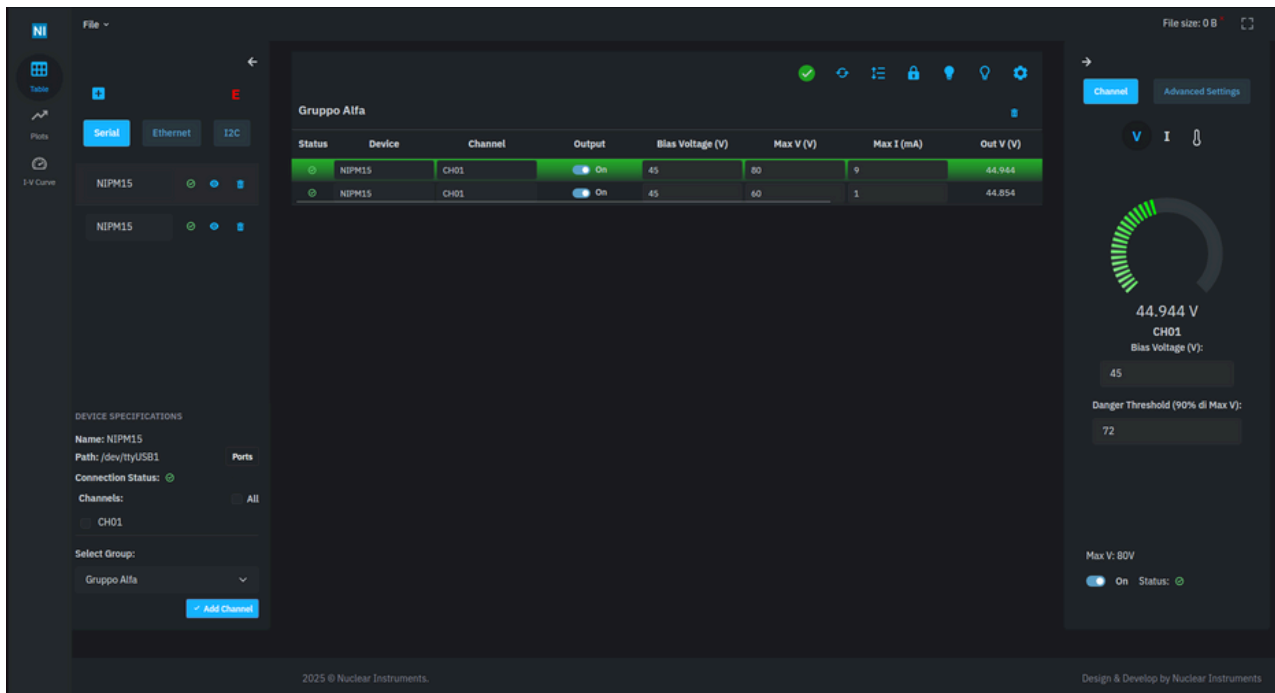
ZEUS è stata progettata per offrire un'interfaccia utente intuitiva, e personalizzabile. Ogni sezione del software risponde a esigenze specifiche per alimentatori, garantendo una gestione semplice e immediata anche per utenti non esperti.

La divisione in sezioni, accompagnata da un design moderno, permette di navigare facilmente tra le varie funzionalità e di avere sempre a portata di mano le informazioni più rilevanti.

IV.IV.VII.I – Tabella di controllo

La pagina della Tabella di controllo rappresenta la sezione principale di ZEUS, progettata per offrire un punto di accesso centralizzato per la gestione e il monitoraggio dei dispositivi.

Per ogni dispositivo e ogni canale, l'interfaccia offre la possibilità di visualizzare in modo chiaro e immediato tutti i parametri operativi fondamentali.



E' possibile accedere a funzioni avanzate e alla configurazione di ciascun canale, adattando le impostazioni in base alle esigenze specifiche.

La sidebar laterale situata sulla sinistra consente di selezionare le connessioni attive e di aggiungere nuovi canali o dispositivi ai diversi gruppi logici.

A destra, un pannello dettagliato consente di monitorare e configurare in modo più approfondito i parametri del canale selezionato.

L'interfaccia è stata progettata con uno stile scuro e

minimalista, pensato per garantire la massima leggibilità e comfort visivo anche durante lunghe sessioni di lavoro.

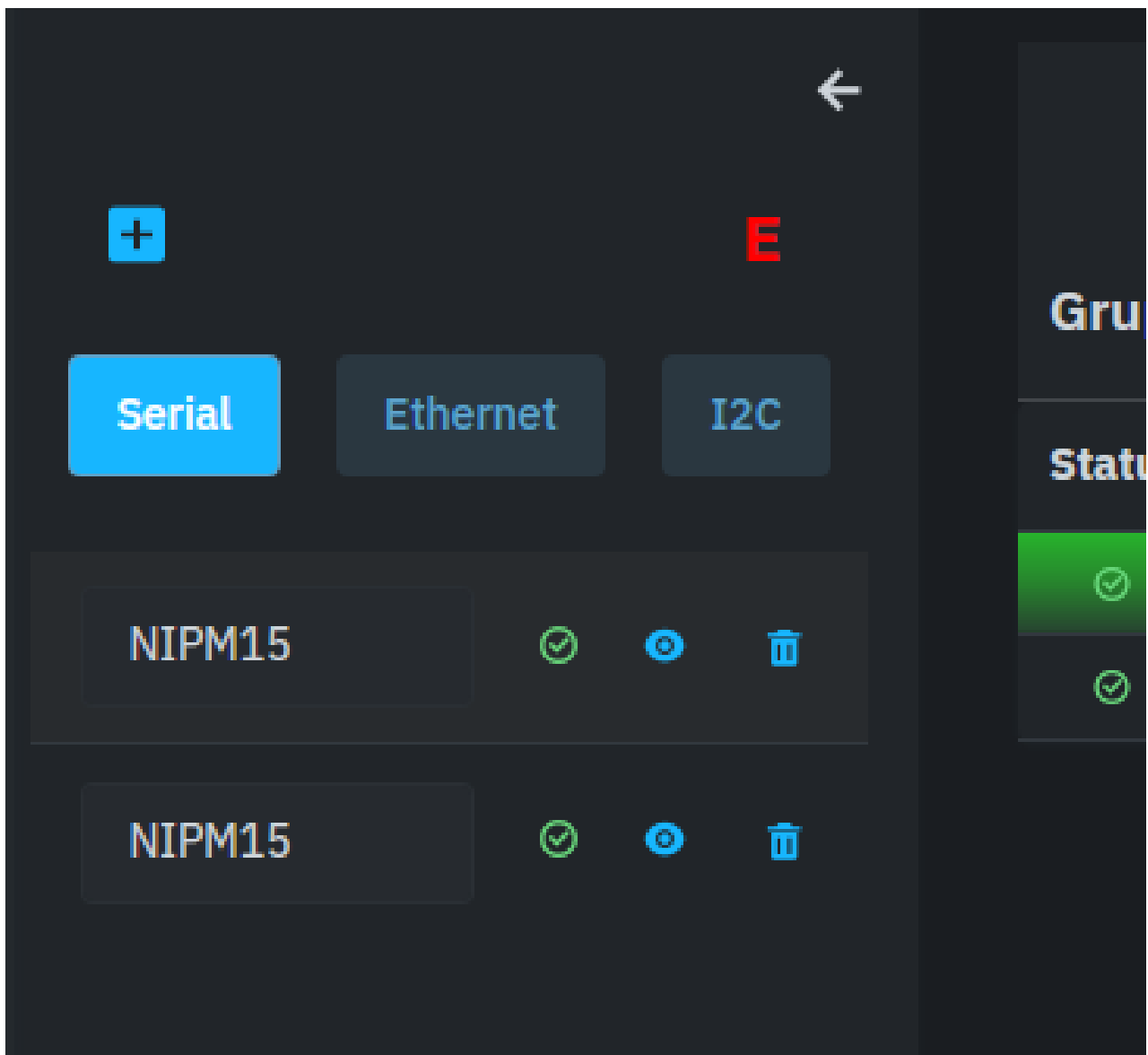
Questa pagina rappresenta il cuore operativo di ZEUS.

IV.IV.VII.I.I – Sidebar devices

La Sidebar devices è stata progettata per offrire agli utenti una panoramica completa di tutti i dispositivi connessi. Situata sul lato sinistro dell'interfaccia, svolge il ruolo di un vero e proprio centro di comando rapido.

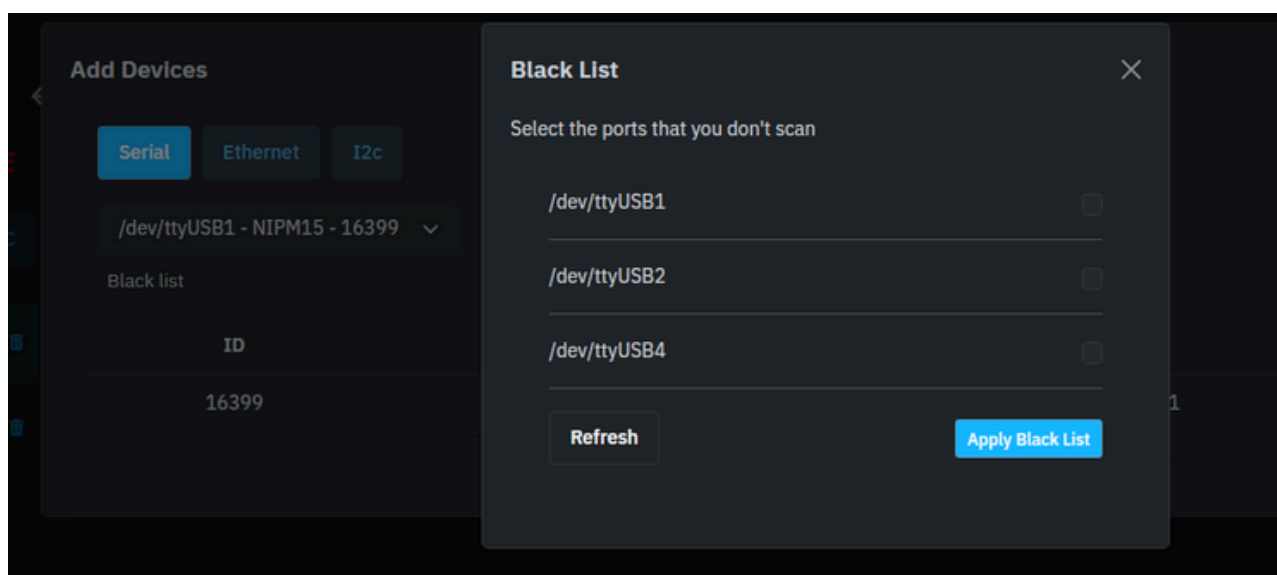
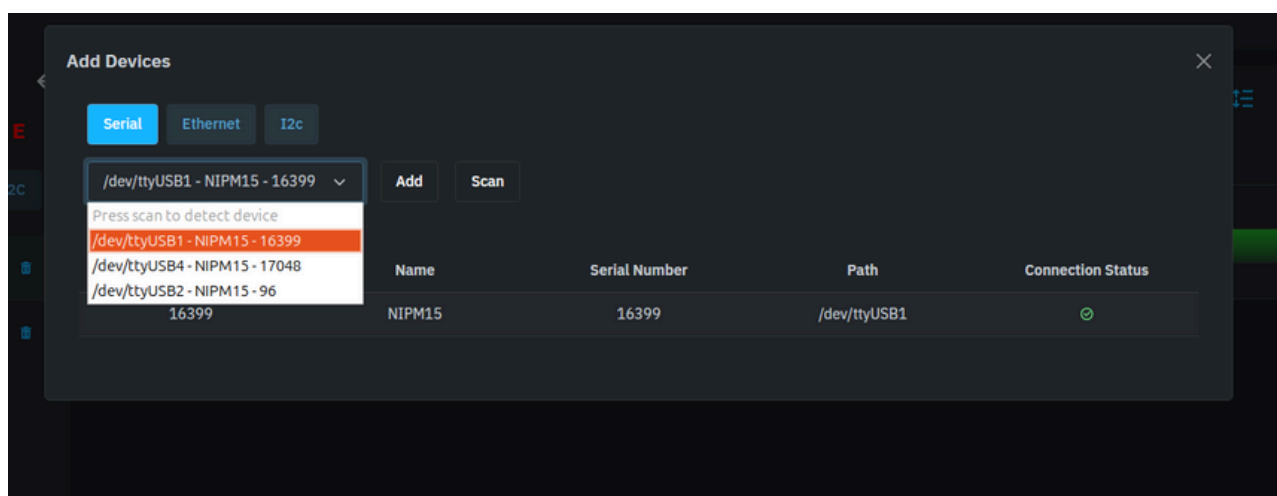
Consente di selezionare il tipo di connessione, che può essere Serial, Ethernet o I²C, in base ai dispositivi collegati disponibili. Oltre a questo, offre la possibilità di visualizzare in modo immediato lo stato di ciascun dispositivo.

La gestione dei dispositivi è resa estremamente semplice, infatti è possibile aggiungerli o rimuoverli utilizzando i pulsanti dedicati.



La Sidebar integra una funzione di scansione per il riconoscimento dei dispositivi, facendo scegliere a chi utilizza il software quando scanisonare le porte per vedere se ci sono dispositivi collegati.

Chiunque utilizza il software ha pure la possibilità di aggiungere delle porte nella Black List. Le porte inserite nella non saranno mai prese in considerazione, così da evitare la comunicazione nel caso ci siano degli altri dispositivi connessi.



La Sidebar Devices consente anche di selezionare i canali disponibili per ogni dispositivo e di associarli a un gruppo logico per poi essere inserito nella tabella principale.

DEVICE SPECIFICATIONS

Name: NIPM15

Path: /dev/ttyUSB1 **Ports**

Connection Status: 

Channels: ☒ All

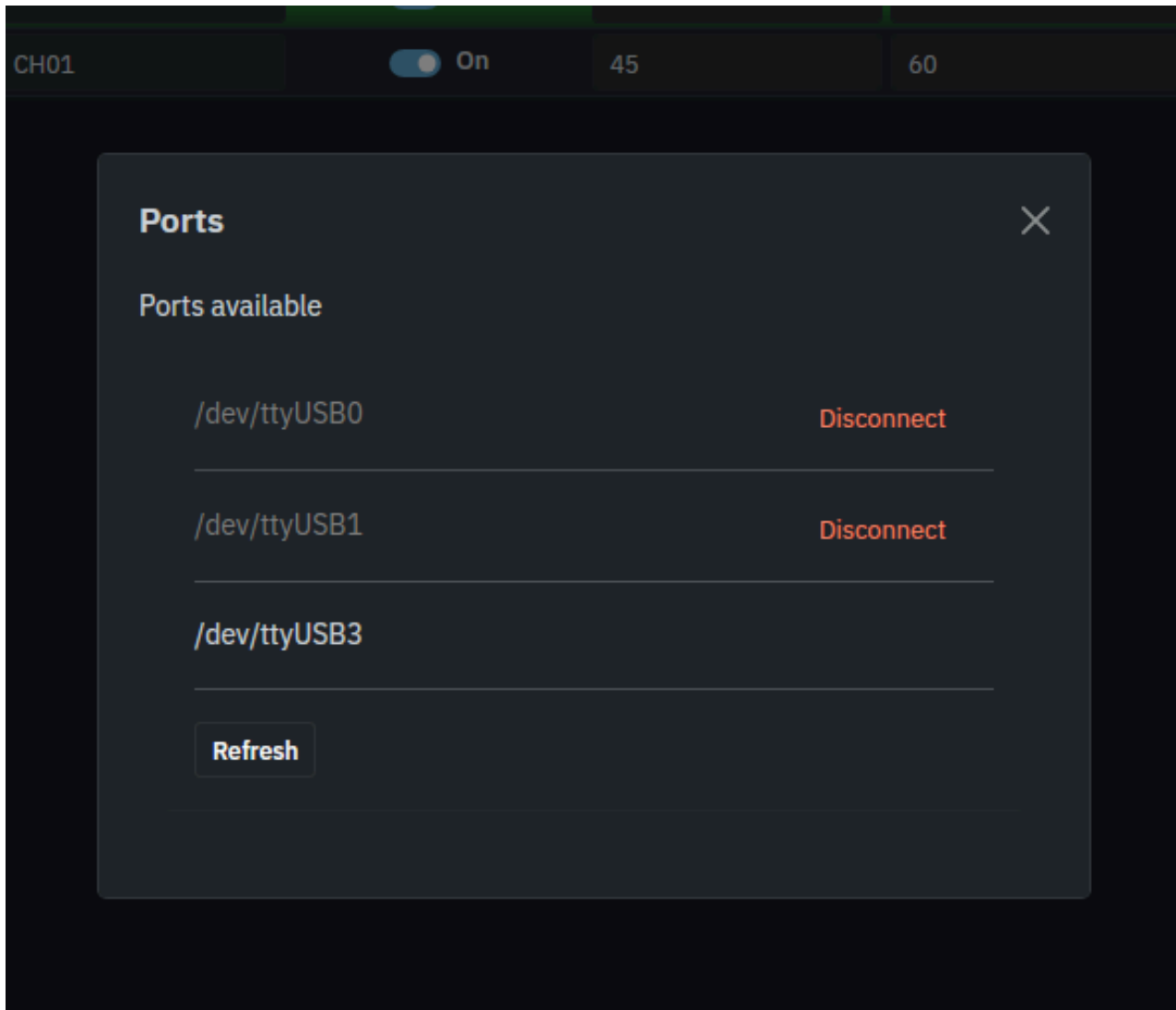
☒ CH01

Select Group:

Gruppo Alfa 

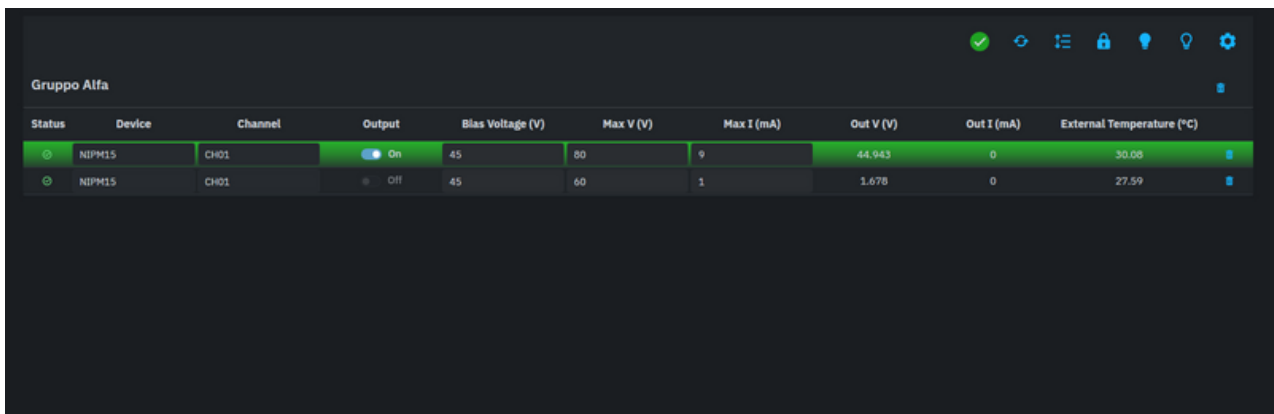
 **Add Channel**

E' presente un pulsante per effettuare il reconnect in caso di disconnessioni del dispositivo, e un pulsante per visionare le porte aperte per eventualmente chiuderle



IV.IV.VII.I.II – Tabella principale

La Tabella principale rappresenta il cuore di ZEUS, offrendo una vista completa e dettagliata delle condizioni di ciascun canale collegato ai dispositivi. Questa tabella è stata progettata con grande attenzione per garantire la massima leggibilità e un controllo immediato su tutti i parametri fondamentali indispensabili per il controllo degli alimentatori.



Status	Device	Channel	Output	Bias Voltage (V)	Max V (V)	Max I (mA)	Out V (V)	Out I (mA)	External Temperature (°C)
⊕	NIPM15	CH01	On	45	80	9	44.943	0	30.08
⊖	NIPM15	CH01	Off	45	60	1	1.678	0	27.59

Per ogni canale nella tabella, viene mostrato lo stato, grazie a un indicatore visivo che segnala se è attivo, disattivo o in fault. Sono riportati il nome del dispositivo e del canale, personalizzabili, a cui si riferiscono i dati, e un pratico interruttore consente di attivare o disattivare rapidamente il canale stesso. Vengono visualizzati anche i principali

parametri operativi, come Bias Voltage (V), Max Voltage (V) e Max Current (mA), insieme ai valori effettivi misurati di Output Voltage (V) e Output Current (mA) e la temperatura esterna (°C).

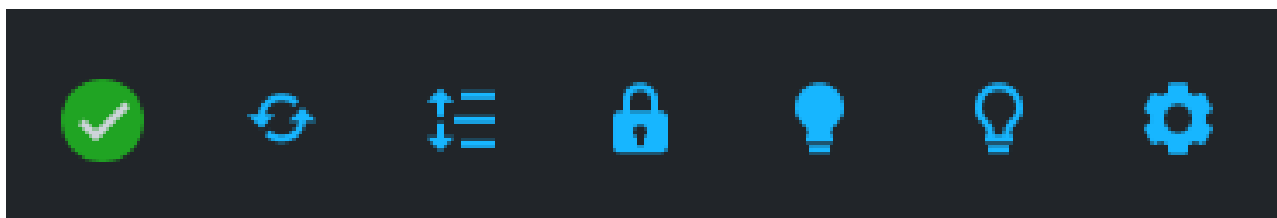
Inoltre ogni canale è associato ad un gruppo di appartenenza dando così un ordine aggiuntivo.

Gruppo Alfa										
Status	Device	Channel	Output	Bias Voltage (V)	Max V (V)	Max I (mA)	Operation Mode	Out V (V)	Out I (mA)	External Temperature
🟢	N3PM15	CH01	On	43.5	80	9	Digital	43.477	0	26.82
🟢	N3PM15	CH01	Off	45	60	1	Digital	1.407	0	26.39

Gruppo Beta										
Status	Device	Channel	Output	Bias Voltage (V)	Max V (V)	Max I (mA)	Operation Mode	Out V (V)	Out I (mA)	External Temperature
🟢	N3PM15	CH01	Off	30	80	9	Digital	1.261	0	30.34

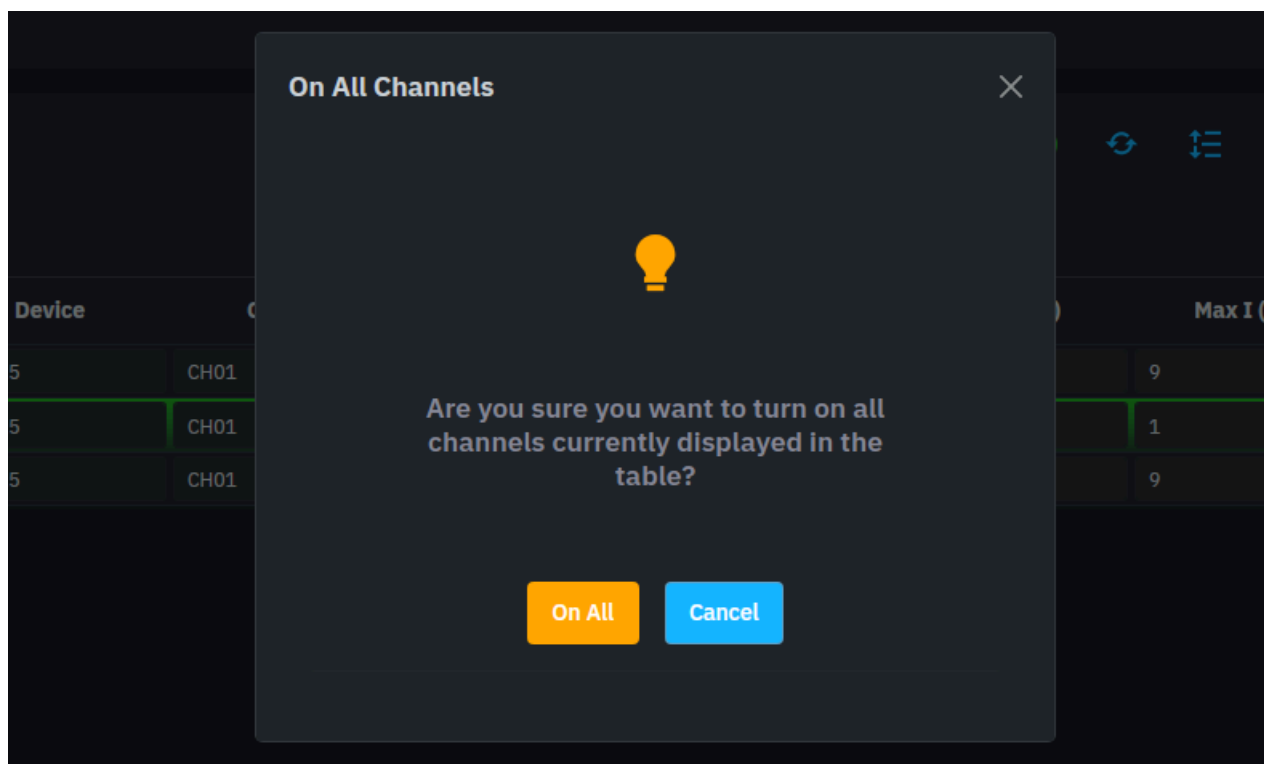
Barra di navigazione e pulsanti di controllo

Sopra la Tabella principale è presente una barra di navigazione che racchiude i principali pulsanti di controllo.



Ogni pulsante è progettato per facilitare operazioni frequenti:

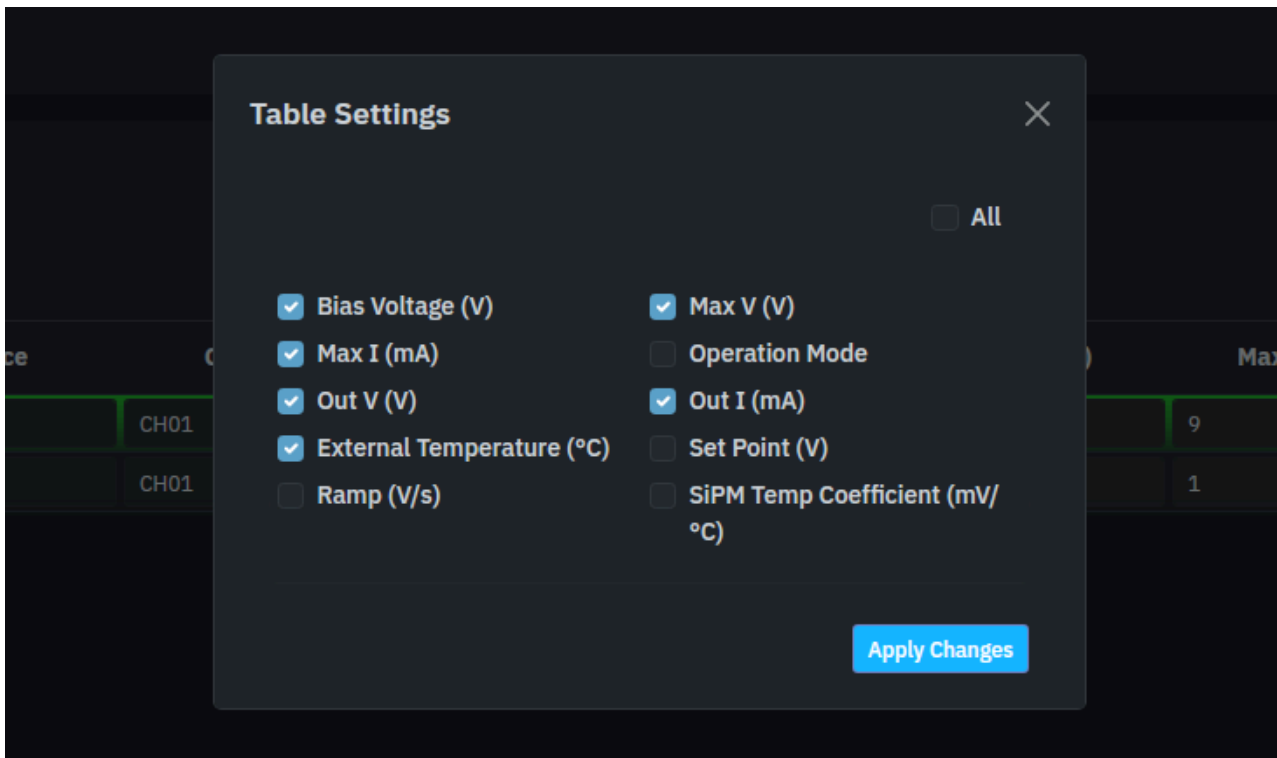
- Indicatore di Edit, fornisce un feedback visivo per segnalare che ci sono delle celle di input in fase di edit. Reconnect all devices, permette di eseguire rapidamente il reconnect di tutti i dispositivi in tabella.
- Ingrandisci righe, consente di aumentare la dimensione delle righe della tabella.
- Lucchetto, abilita l'edit dei parametri se aperto per una maggiore sicurezza.
- Accendi tutti i devices, comando per accendere simultaneamente tutti i canali visibili nella tabella
- Spegni tutti i devices, al contrario, permette di spegnere tutti i canali visibili.
- Settings della tabella apre il menu di configurazione della tabella, dove è possibile selezionare quali colonne visualizzare.



Personalizzazione tabella

La Tabella principale di ZEUS offre un sistema di personalizzazione avanzata che consente di adattare la visualizzazione alle specifiche esigenze dell'utente e al tipo di operazione da svolgere.

Attraverso il pannello di configurazione, accessibile tramite l'icona Settings, l'utente ha la possibilità di scegliere quali colonne visualizzare. Questo pannello permette di attivare o disattivare i campi di interesse, creando così una visualizzazione su misura.



Un'altra possibilità di personalizzazione riguarda i nomi dei dispositivi e dei gruppi. L'utente può assegnare nomi personalizzati, facilitando così l'organizzazione logica dei dispositivi.

Infine, la tabella offre anche la possibilità di regolare l'altezza delle righe, grazie all'icona di ingrandimento è possibile aumentare lo spazio verticale delle righe, facilitando la lettura dei valori e l'editing.

Gruppo Alfa

Status	Device	Channel	Output	Bias Voltage (V)	Max V (V)	Max I (mA)	Out V (V)	Out I (mA)	External Temperature (°C)
⊖	NIPM15	CH01	On	45	80	9	44.942	0	30.38
⊖	NIPM15	CH01	On	45	60	1	44.845	0	27.49

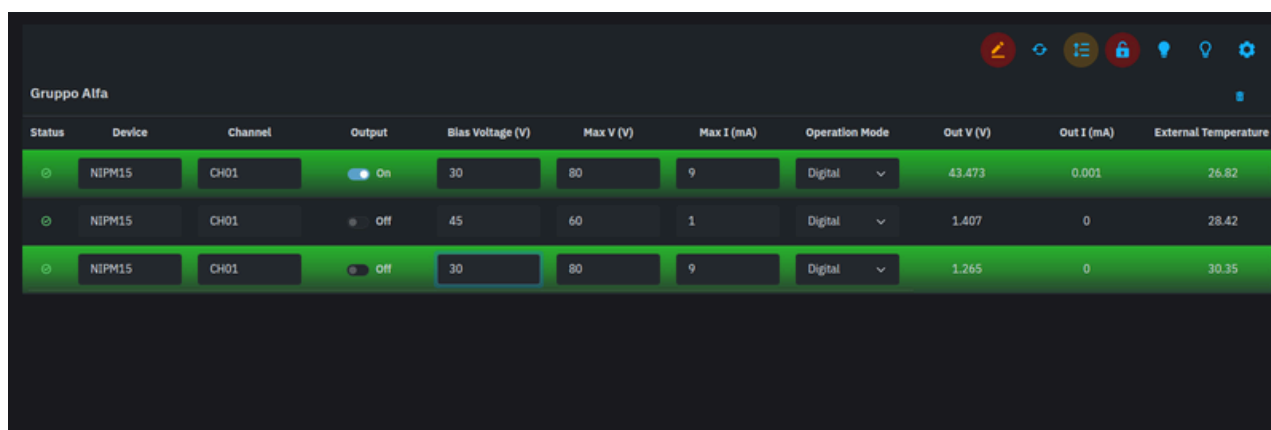
Edit multiplo dei parametri

Oltre alla personalizzazione, è stata anche implementata una funzione avanzata di modifica multipla dei parametri, che richiama lo stile dei classici fogli di calcolo come Excel.

Questa modalità consente all'utente di selezionare e modificare simultaneamente più celle, rendendo molto più semplice e veloce la gestione di operazioni ripetitive.

Utilizzando i tasti Ctrl e Shift durante la selezione, è possibile evidenziare più celle contemporaneamente.

Una volta effettuata la selezione, l'utente può applicare una modifica a un campo specifico e questa modifica verrà automaticamente propagata a tutti i dispositivi o canali selezionati.



Gruppo Alfa

Status	Device	Channel	Output	Bias Voltage (V)	Max V (V)	Max I (mA)	Operation Mode	Out V (V)	Out I (mA)	External Temperature
⊖	NIPM15	CH01	On	30	80	9	Digital	43.473	0.001	26.82
⊖	NIPM15	CH01	Off	45	60	1	Digital	1.407	0	28.42
⊖	NIPM15	CH01	Off	30	80	9	Digital	1.265	0	30.35

Per facilitare ulteriormente l'inserimento dei dati, il tasto Tab consente di spostarsi rapidamente alla cella sottostante.

Quando l'utente ha terminato la modifica o decide di annullarla, può semplicemente premere il tasto Esc per uscire dalla modalità di editing e tornare alla visualizzazione standard della tabella.

IV.IV.VII.I.III – Sidebar Channel

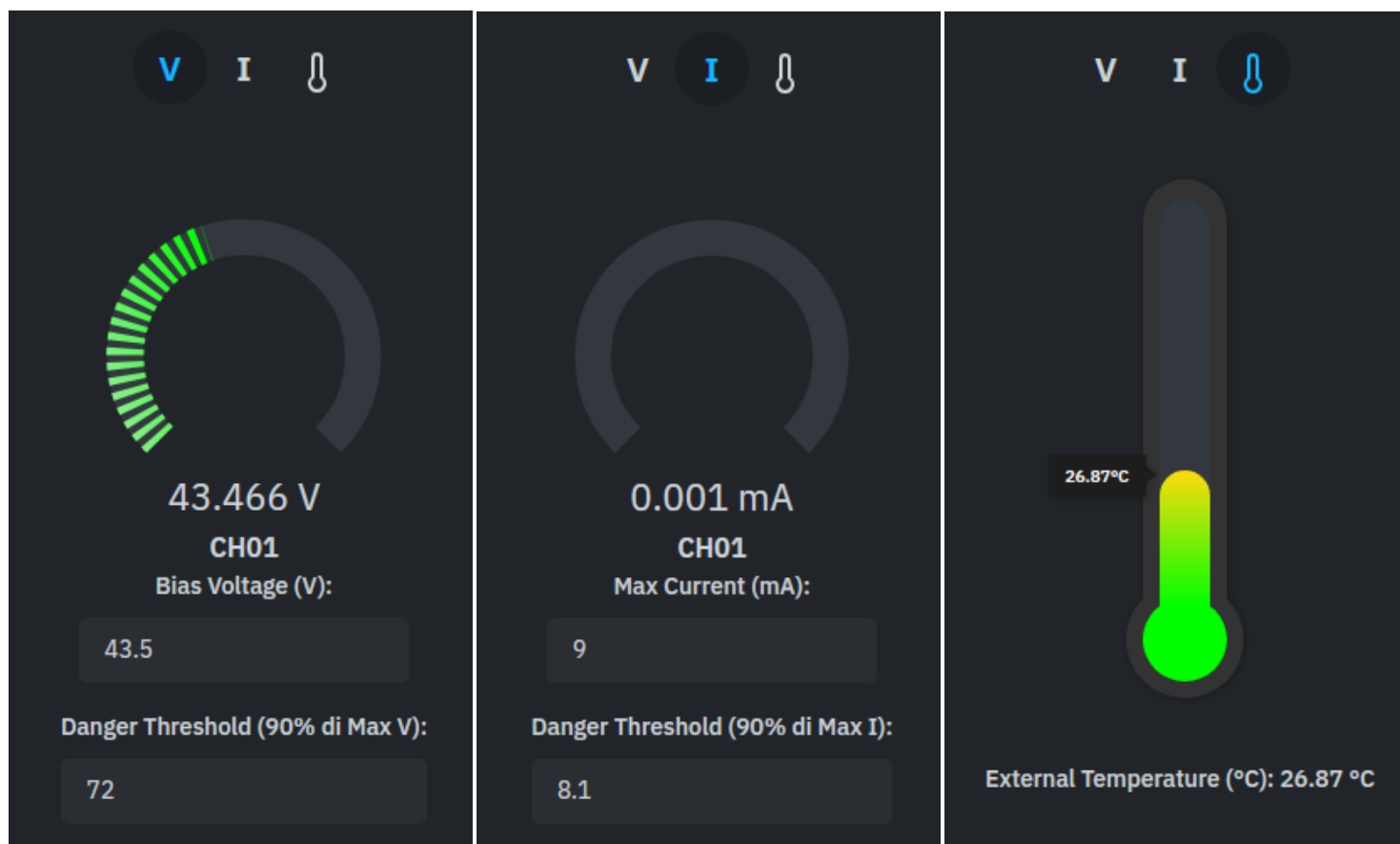
La Sidebar Channel rappresenta un componente particolarmente versatile all'interno della Tabella di controllo di ZEUS.

È stata progettata per offrire all'utente una panoramica dettagliata in tempo reale dei parametri operativi del canale selezionato.

La barra laterale è suddivisa in due sezioni principali: Channel e Advanced Settings.

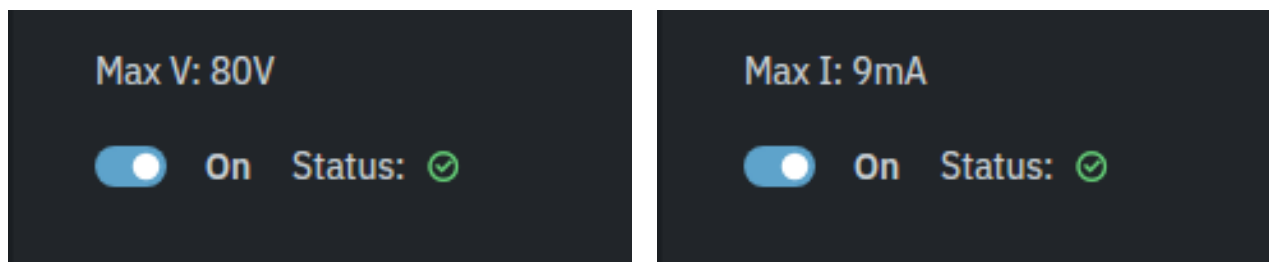
Sezione Channel

All'interno della sezione Channel, vengono visualizzati i valori istantanei dei parametri principali selezionati, come il Bias Voltage, il Max Current o la Temperatura. Questi valori vengono mostrati tramite componenti grafici. La sezione include anche i valori di soglia, come il Danger Threshold, calcolato come percentuale del valore massimo consentito, per fornire un riferimento di sicurezza in tempo reale.



Sono presenti campi editabili che consentono di modificare direttamente i limiti operativi del canale, come il Bias Voltage il Max Current, semplificando così eventuali aggiustamenti.

Il toggle switch per accendere o spegnere il canale e l'indicatore dello stato di connessione completano la sezione, fornendo un controllo sempre a portata di mano.



Advanced Settings

La sezione Advanced Settings permette di intervenire su parametri più specifici.

La sottosezione General fornisce informazioni generali, come il Serial Number, la Firmware Version, la Hardware Version e l'I2C Base Address, offrendo un quadro completo della configurazione del dispositivo.

La parte dedicata alla Temperature consente di gestire le impostazioni legate al sensore di temperatura selezionato, includendo parametri avanzati della formula di calcolo e la modalità di feedback.

Infine, la sezione dedicata al Low pass filter permette di configurare filtri per stabilizzare le misure, come quelle relative all'Output Voltage, all'Output Current e alla Temperatura, migliorando così la precisione dei dati acquisiti.

→

Channel Advanced Settings

ADVANCED SETTINGS

General Temperature Low pass filter

Serial Number: 16399

Firmware Version: 6.2

Hardware Version: 2

I2C Base Address: 0x70

Change I2C Base Address

Ramp Speed (V/s): 10

Save all settings on EEPROM

→

Channel Advanced Settings

ADVANCED SETTINGS

General Temperature Low pass filter

Sensor: TMP37

CTemp parameter

M₂: 0

M: 50

Q: 0

Apply

$$T [^{\circ}\text{C}] = M_2 \cdot V^2 + M \cdot V + Q$$

SiPM Temp Coefficient (mV/°C): 0

Temperature Feedback Mode: SiPM Te

→

Channel Advanced Settings

ADVANCED SETTINGS

General Temperature Low pass filter

Output V Filter: 0

Output I Filter: 0

Reference V Filter: 0

Temperature Filter: 0

La logica di configurazione e visualizzazione delle impostazioni avanzate di ciascun dispositivo è stata progettata in modo estremamente flessibile. Questa flessibilità è garantita dall'utilizzo di file JSON esterni, che definiscono in modo strutturato campi e parametri disponibili per ciascun dispositivo. Questa scelta consente di adattare la gestione dei parametri ai diversi tipi di dispositivi collegati, senza dover modificare la logica dell'interfaccia.

```
{
  "fields": [
    {
      "key": "store_on_flash",
      "label": "Save all settings on EEPROM",
      "type": "Bigbutton",
      "value": true,
      "readonly": false
    }
  ]
},
{
  "name": "temperature",
  "order": 2,
  "fields": [
    {
      "key": "temperature_sensor",
      "label": "Sensor",
      "type": "select",
      "readonly": false,
      "default": "TMP37",
      "options": [
        "TMP37",
        "LM94021",
        "Other"
      ]
    },
    {
      "key": "labelCTemp",
      "label": "CTemp parameter",
      "type": "label"
    }
  ]
},
```

```
const renderField = (field) => {  
  </div>  
  );  
  case "label":  
    return (  
      <div key={key} className="mt-4">  
        <label>{label}</label>  
        <span className="text-muted">{def}</span>  
      </div>  
    );  
  case "select":  
    return (  
      <div key={key} className="d-flex align-items-center justify-content-between mb-2">  
        <label style={{ paddingTop: "5px" }}>{label}</label>  
        <select  
          value={fieldValues[field.key] ?? ""}  
          onChange={(e) => {  
            const newValue = e.target.value;  
            setFieldValues(prev => ({  
              ...prev,  
              [field.key]: newValue  
            }));  
          }}  
          className="form-select form-select-sm"  
          style={{ maxWidth: "40%", fontWeight: "lighter" }}  
        >  
          {(field.options || []).map((opt) => (  
            <option key={opt} value={opt}>{opt}</option>  
          ))}  
        </select>  
      </div>  
    );  
  };  
};
```

IV.IV.VII.II – Plot in tempo reale

Il Plot in tempo reale, consente di monitorare e analizzare in diretta i parametri fondamentali dei canali collegati, offrendo uno strumento avanzato per la diagnosi e il controllo dei dispositivi ad alta tensione.



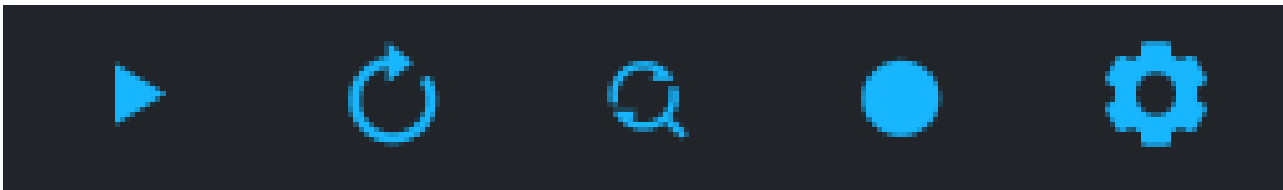
IV.IV.VII.II.I – Sidebar Channels

La Sidebar Channels nella sezione dedicata al plot in tempo reale di ZEUS rappresenta uno strumento fondamentale per il monitoraggio mirato dei canali.

Collocata sulla sinistra dell'interfaccia grafica, è stata concepita per offrire una panoramica completa di tutti i gruppi e i canali attivi. In particolare, la lista dei gruppi permette di organizzare logicamente i canali.

Barra di controllo rapida

La barra di controllo nella sezione del plot in tempo reale di ZEUS offre cinque pulsanti principali, ognuno con un ruolo specifico e ben definito per agevolare l'utente nella gestione dei grafici e nella registrazione dei dati.

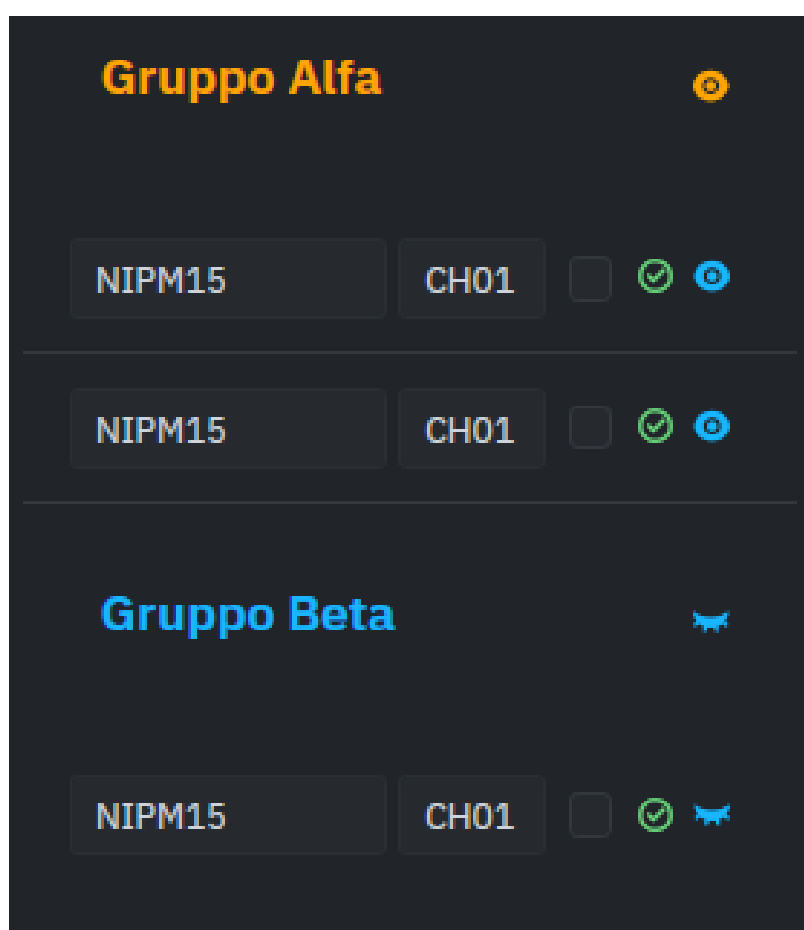


- Start/Stop, ha la funzione di avviare l'arrivo dei punti nel grafico, premendolo si attiva la visualizzazione in tempo reale dei dati.
- Restart, serve a resettare i punti nel grafico, visualizzati fino a quel momento.
- Resize, consente di eseguire un'operazione di resize o zoom automatico del grafico, particolarmente utile per tornare ai valori di scala impostati precedentemente.
- Record, permette di avviare la registrazione su file, i dati in arrivo vengono salvati in un file, così da poter essere consultati o analizzati anche successivamente.
- Record Settings, apre un modal dedicato alle impostazioni avanzate, così da poter configurare in modo dettagliato i parametri relativi alla registrazione, adattando il tutto alle esigenze specifiche di ciascuna acquisizione.

Struttura e funzionalità

Per ciascun canale, sono presenti indicatori visivi che segnalano lo stato di connessione, mostrando se il canale è attivo o meno o addirittura se è in stato di fault, lo stato di visualizzazione, indicando se i dati del canale sono attualmente rappresentati nel grafico e una check-box per la registrazione su file.

Un aspetto particolarmente utile è il controllo diretto della visualizzazione per ogni canale, che consente di attivare o disattivare la sua rappresentazione nel grafico in tempo reale.



IV.IV.VII.II.II – Plot

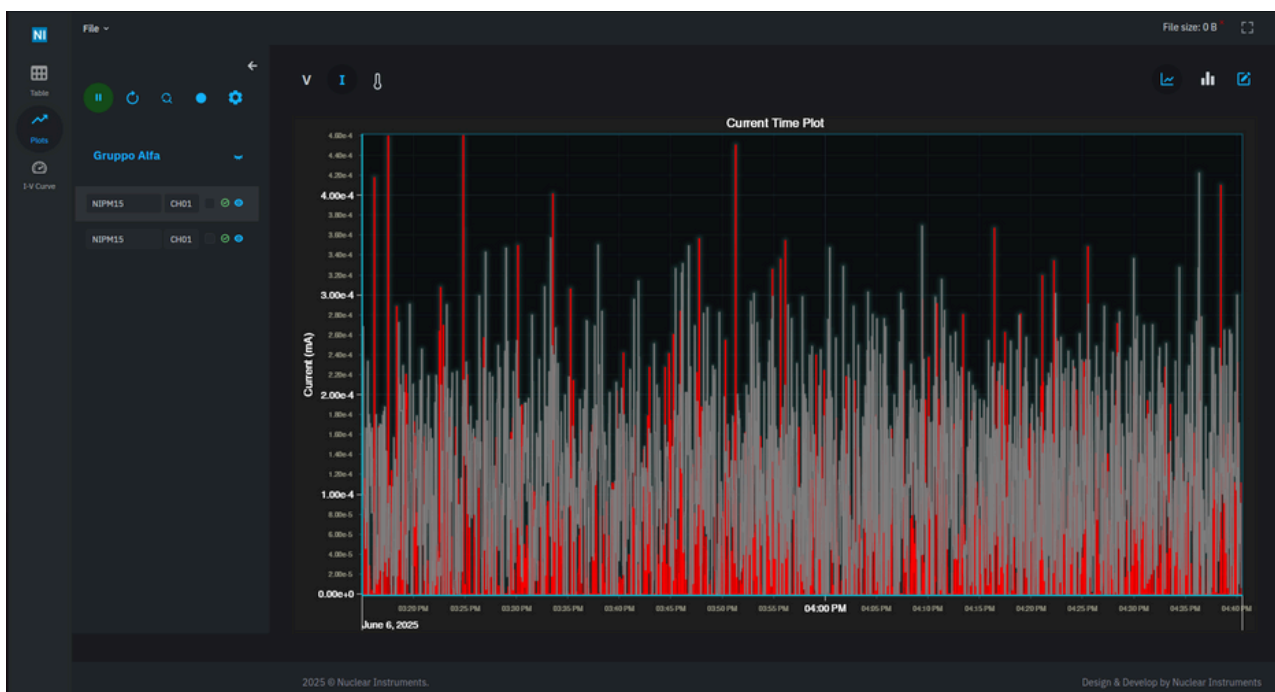
La sezione dedicata ai Plot si divide in due categorie principali il Time Plot, che mostra l'andamento dei parametri principali nel tempo e l'Histogram Plot, che offre una rappresentazione statistica delle distribuzioni dei dati raccolti.

IV.IV.VII.II.II.I – Time Plot

I Time Plot consentono agli utenti di osservare l'evoluzione dei parametri nel tempo avendone la possibilità di alternare la visualizzazione dei dati tra tensione, corrente e temperatura.



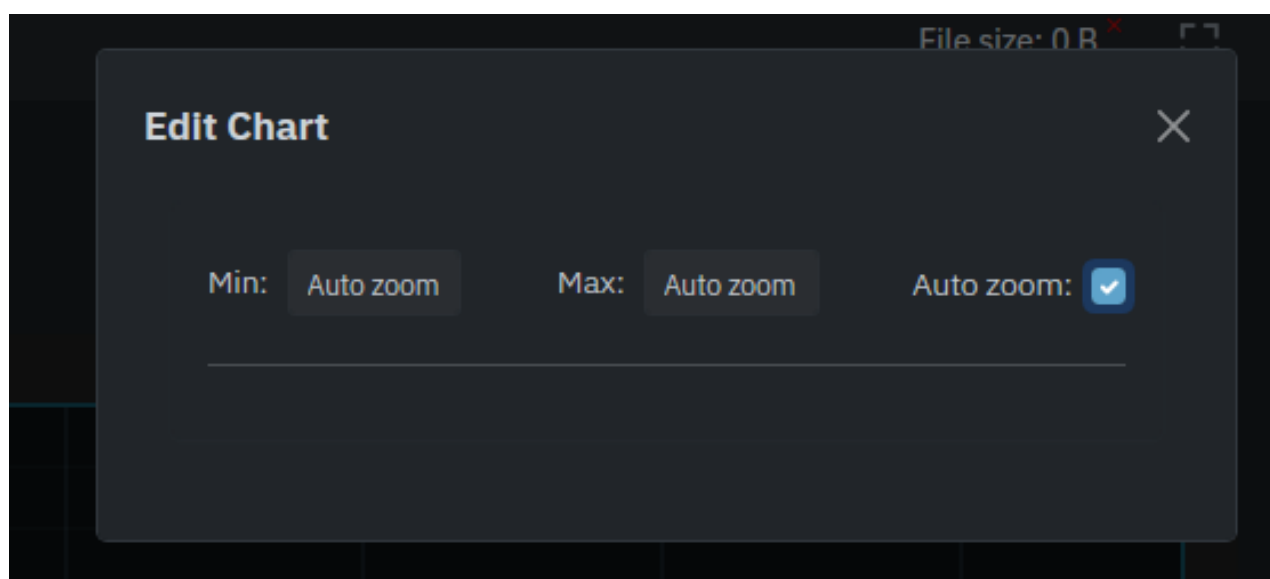
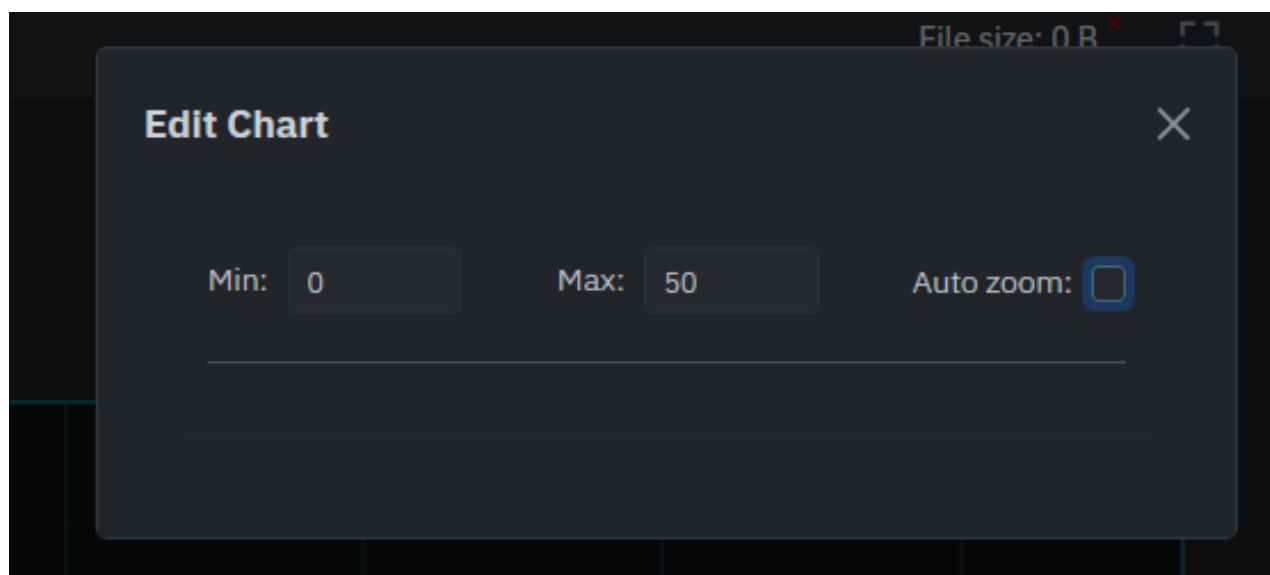
Un aspetto distintivo e particolarmente utile è la modalità di focus, passando con il mouse su una linea o su un canale nella Sidebar Channels, la linea selezionata si illumina e le altre diventano grigie, consentendo di concentrarsi su una singola curva.



Il cursore di lettura permette di visualizzare il valore esatto in un determinato momento temporale, sia per l'asse X (tempo) sia per l'asse Y (valore del parametro).

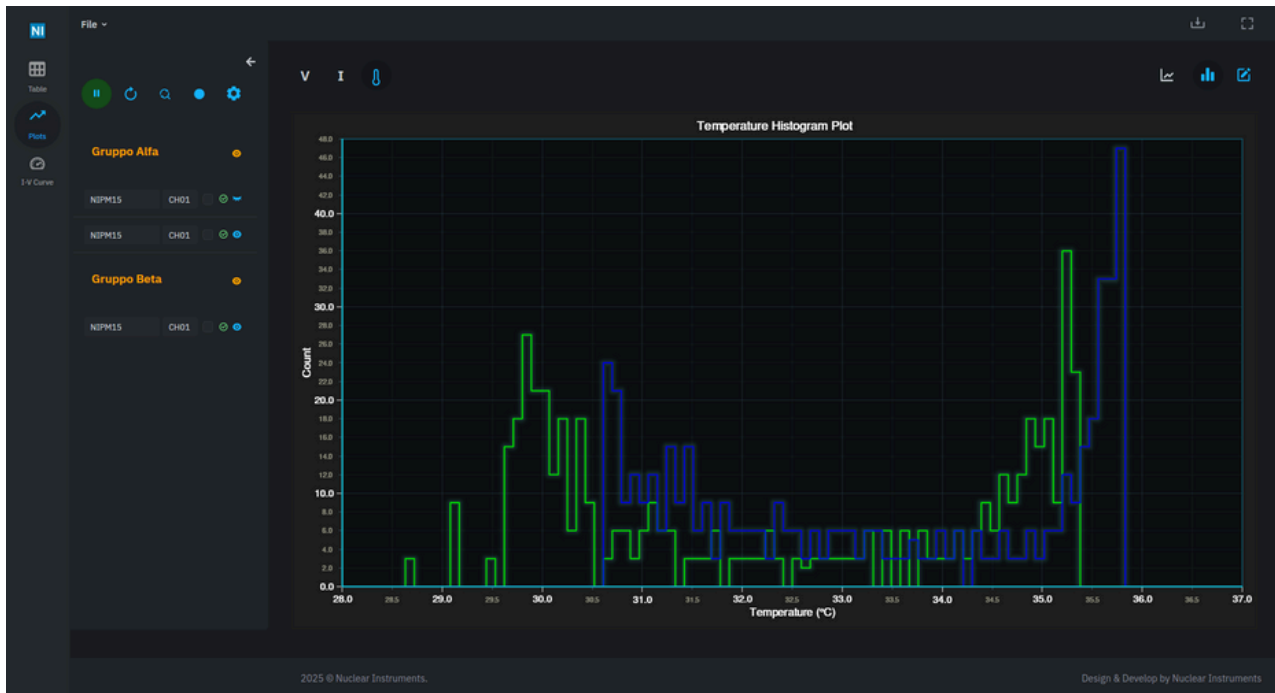


Inoltre, attraverso la finestra di dialogo “Edit Chart”, l’utente può impostare manualmente i limiti minimi e massimi dell’asse Y (Min e Max), adattando la scala in base alle proprie esigenze operative, oppure abilitare l’auto zoom, per un fit automatico del grafico in base ai punti che arrivano man mano nel grafico.



IV.IV.VII.II.II.II – Histogram Plot

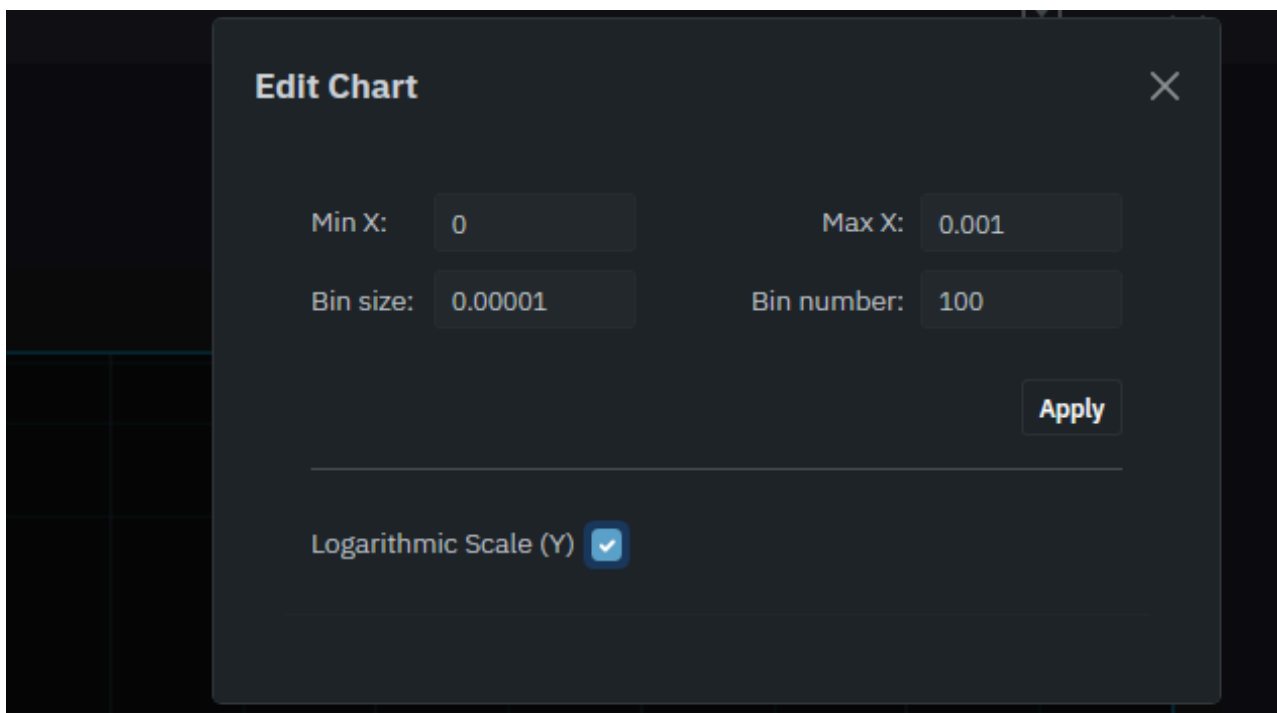
La sezione dedicata agli Histogram Plot, è particolarmente utile per analizzare in modo chiaro, la frequenza dei valori misurati e per identificare eventuali pattern o anomalie che potrebbero emergere durante le sessioni di monitoraggio.



Anche qui c'è la possibilità di personalizzare in modo dettagliato la visualizzazione.

Chiunque può impostare manualmente i limiti minimi e massimi dell'asse X (Min X e Max X), adattando la scala del grafico alle proprie esigenze operative e definire la dimensione e il numero dei bin.

Per i casi in cui i valori hanno ampie variazioni di frequenza, è disponibile anche la modalità a scala logaritmica per l'asse Y (Logarithmic Scale (Y)).



Edit Chart X

Min X: 0 Max X: 0.001

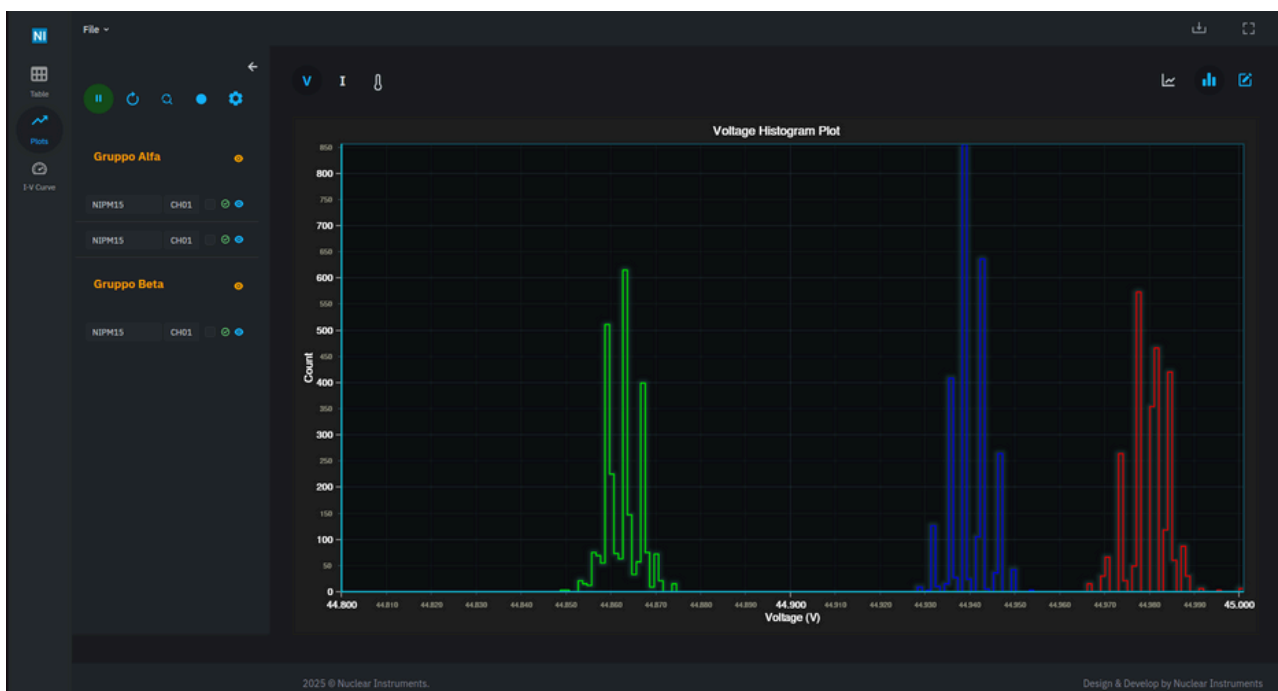
Bin size: 0.00001 Bin number: 100

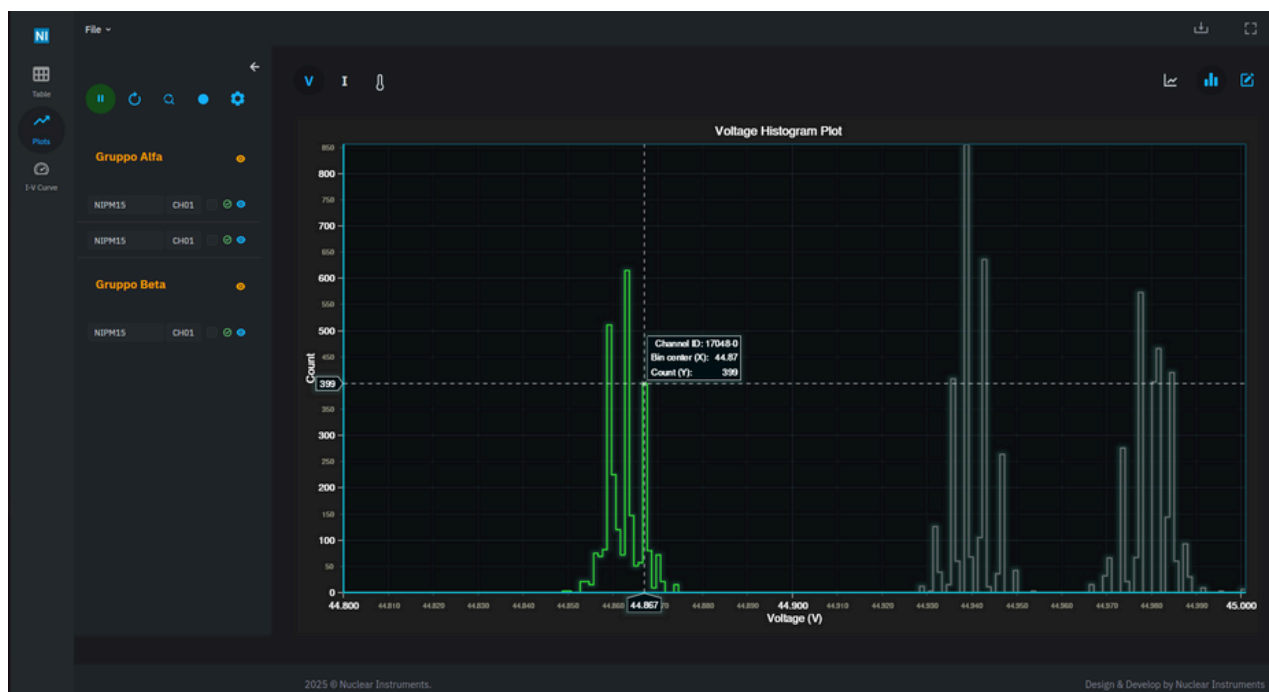
Apply

Logarithmic Scale (Y) ☒



Anche qui, quando il cursore si sposta sopra una barra, vengono mostrati dettagli puntuali come il valore centrale del bin e la frequenza corrispondente.

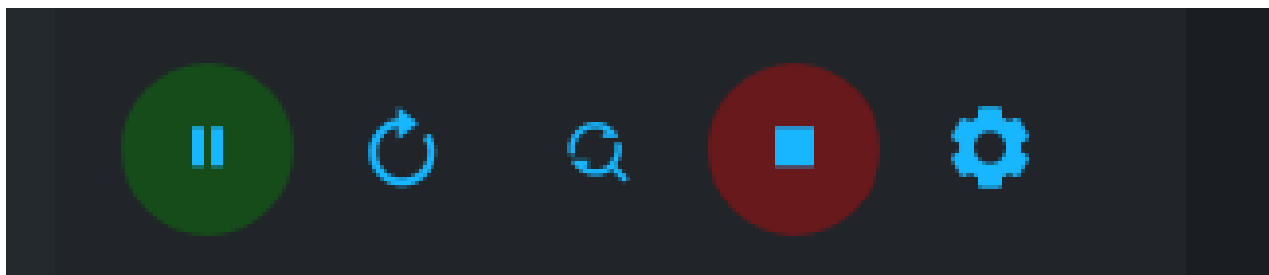




IV.IV.VII.II.III – Registrazione su file

Zeus offre la possibilità di catturare, archiviare ed esportare i dati provenienti dai dispositivi in modo da poterli analizzare attraverso script esterni.

Grazie al backend in go, è possibile salvare in tempo reale i dati sul disco, offrendo maggior sicurezza in caso di crash del software, tutti i dati infatti saranno salvati fino al crash.

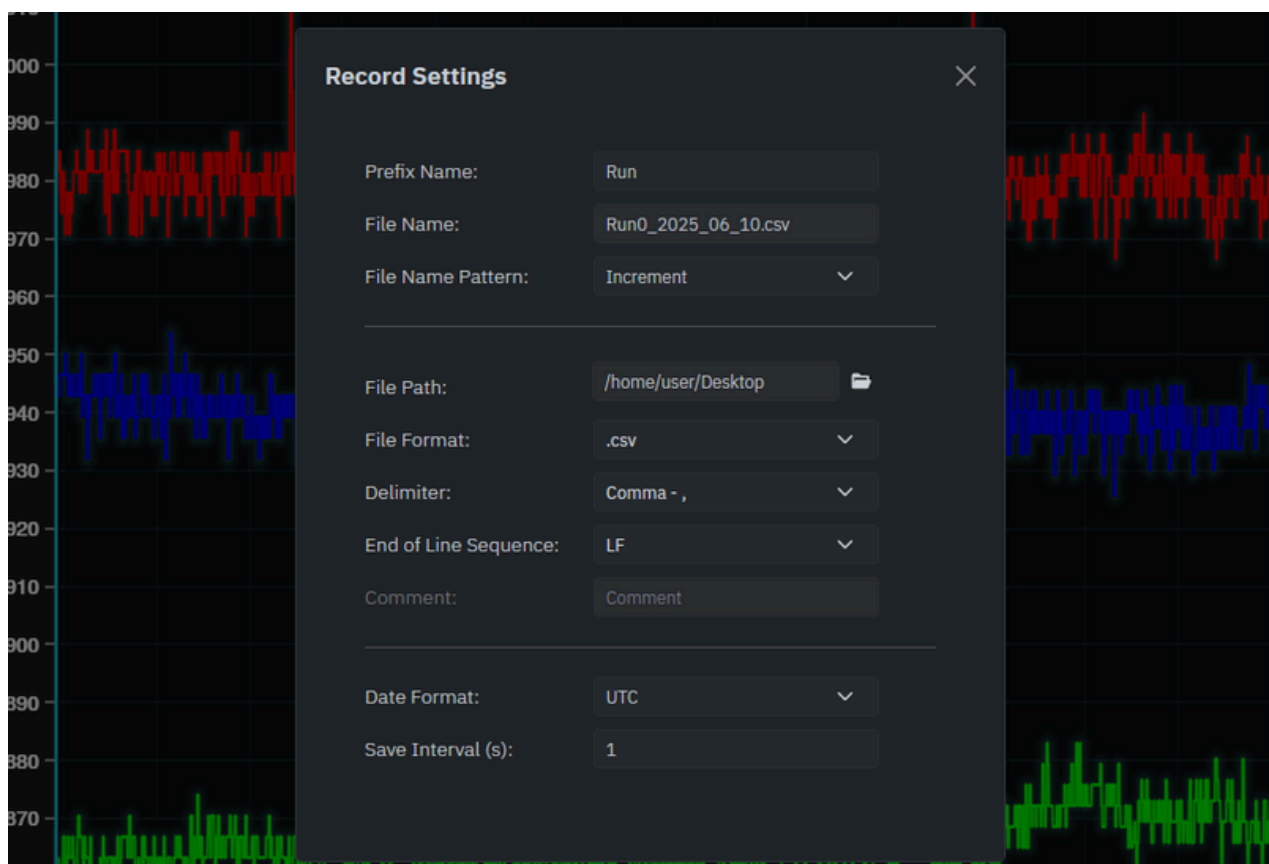


Al momento dell'avvio l'utente può scegliere quali canali vuole registrare e quali no.

La registrazione può essere configurata tramite una sezione dedicata all'interno della barra laterale dell'interfaccia. In questa sezione, si ha la possibilità di scegliere diversi parametri per personalizzare al massimo il salvataggio dei dati.

Ogni dato, per quanto riguarda i Time Plot, viene appeso all'ultimo registrato.





È possibile, definire un prefisso per i file generati che può essere generato automaticamente dal sistema o impostato manualmente a seconda delle esigenze. Si può anche, attivare la modalità di incremento, che consente di generare file con nomi progressivi, come Run0, Run1 e così via.

Record Settings ✕

Prefix Name:

File Name:

File Name Pattern:

Record Settings ✕

Prefix Name:

File Name:

File Name Pattern:

Record Settings ✕

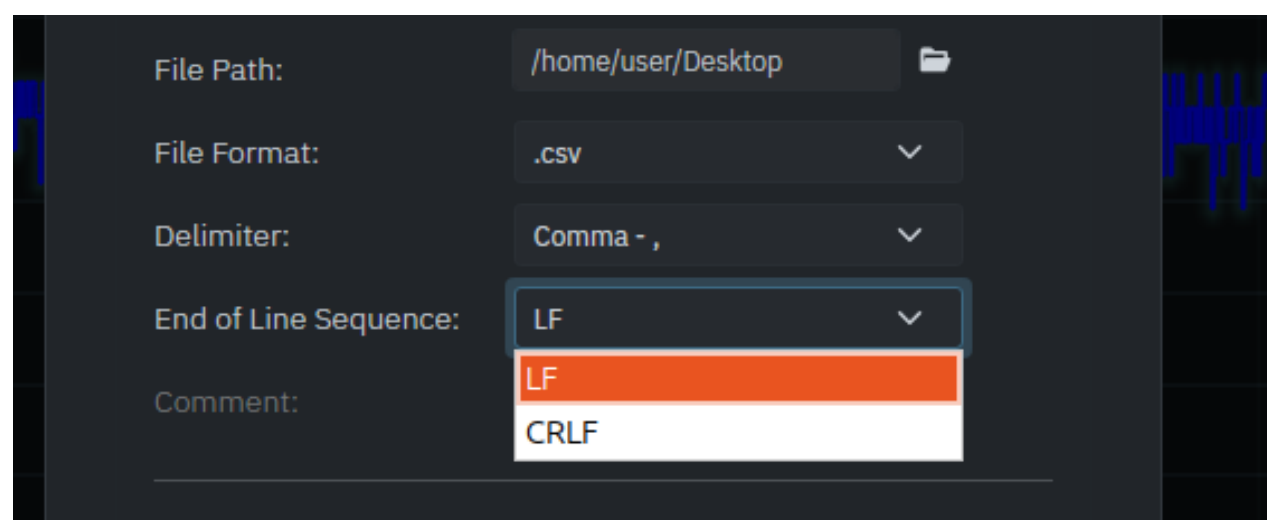
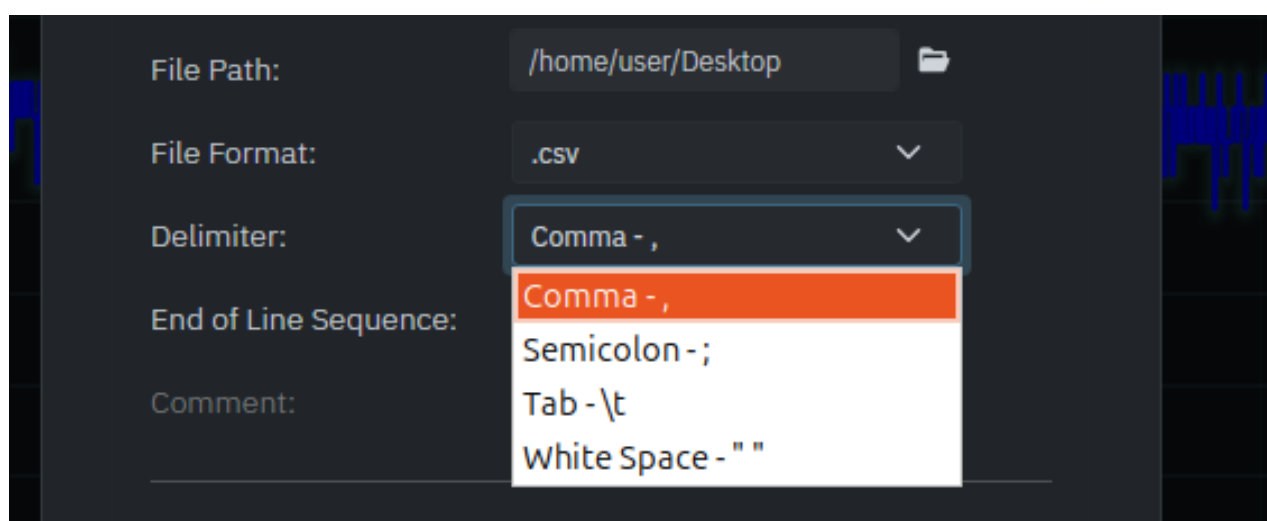
Prefix Name:

File Name:

File Name Pattern:

Altre impostazioni importanti riguardano il percorso di salvataggio, che può essere scelto liberamente all'interno della struttura di cartelle a disposizione, e il formato del file, attualmente supportato in .csv o .jsonl.

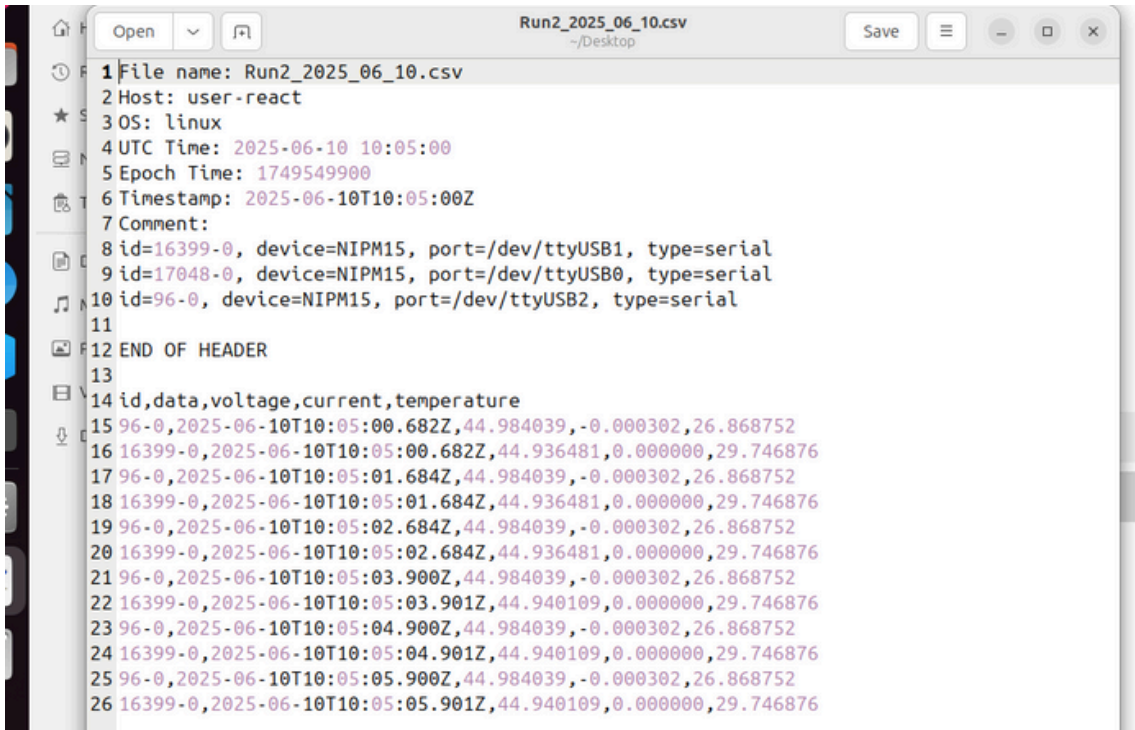
Per garantire la massima flessibilità, l'utente può anche selezionare il delimitatore da utilizzare nei file (virgola, punto e virgola, tab e white space), e personalizzare la sequenza di fine riga (LF o CRLF).



È presente anche un campo commento dove aggiungere note di contesto o descrizioni, utili per documentare al meglio le sessioni di acquisizione.

Un'altra opzione significativa è la possibilità di impostare la frequenza di salvataggio dei dati, tramite il parametro Save Interval (s), che permette di definire ogni quanto i dati devono essere salvati nel file.

I file generati, se scelto, possono contenere anche un'header contenente i dati fondamentali sull'acquisizione.

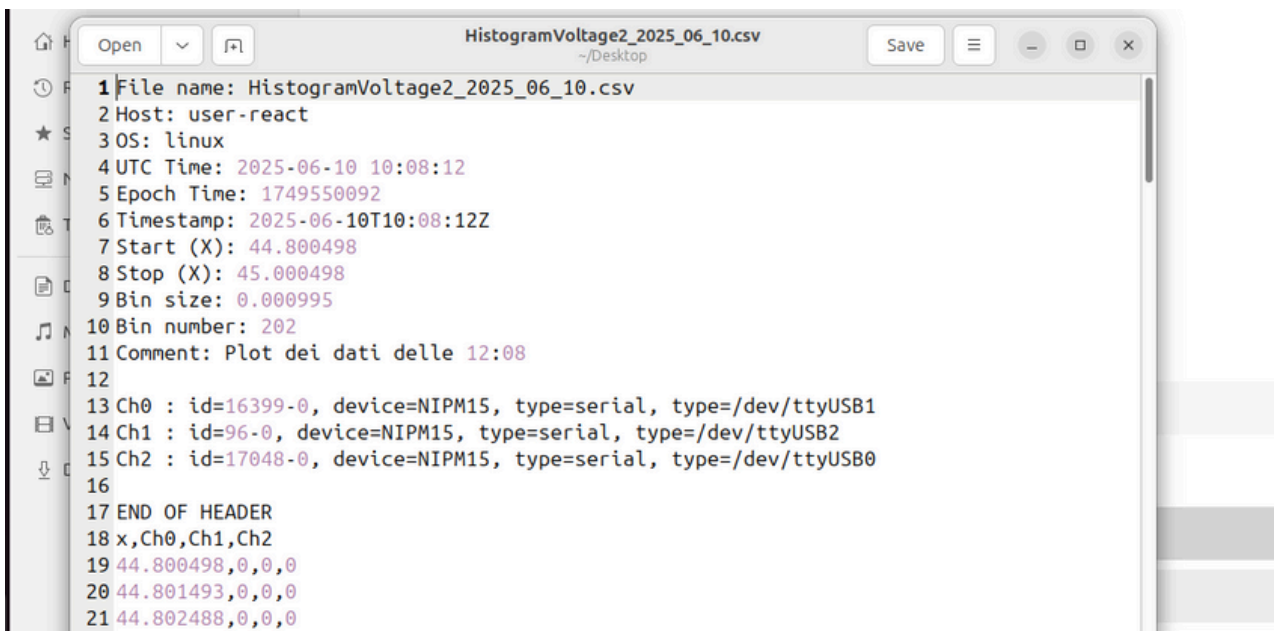
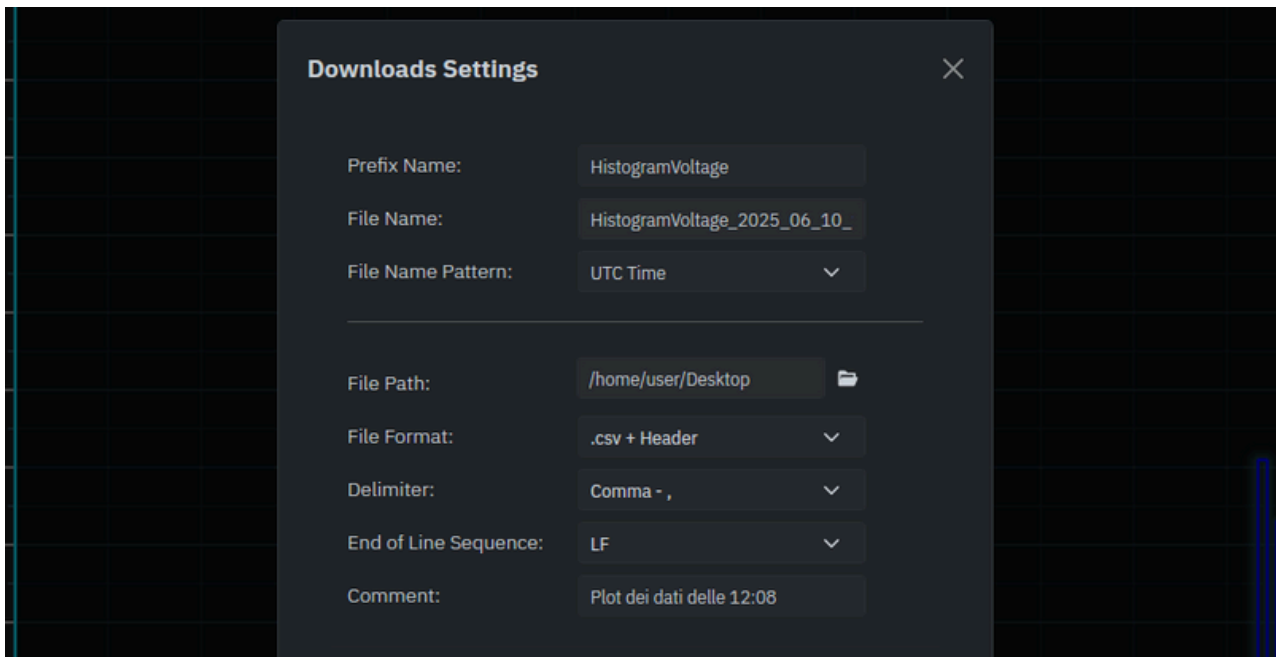


```
1 File name: Run2_2025_06_10.csv
2 Host: user-react
3 OS: linux
4 UTC Time: 2025-06-10 10:05:00
5 Epoch Time: 1749549900
6 Timestamp: 2025-06-10T10:05:00Z
7 Comment:
8 id=16399-0, device=NIPM15, port=/dev/ttyUSB1, type=serial
9 id=17048-0, device=NIPM15, port=/dev/ttyUSB0, type=serial
10 id=96-0, device=NIPM15, port=/dev/ttyUSB2, type=serial
11
12 END OF HEADER
13
14 id,data,voltage,current,temperature
15 96-0,2025-06-10T10:05:00.682Z,44.984039,-0.000302,26.868752
16 16399-0,2025-06-10T10:05:00.682Z,44.936481,0.000000,29.746876
17 96-0,2025-06-10T10:05:01.684Z,44.984039,-0.000302,26.868752
18 16399-0,2025-06-10T10:05:01.684Z,44.936481,0.000000,29.746876
19 96-0,2025-06-10T10:05:02.684Z,44.984039,-0.000302,26.868752
20 16399-0,2025-06-10T10:05:02.684Z,44.936481,0.000000,29.746876
21 96-0,2025-06-10T10:05:03.900Z,44.984039,-0.000302,26.868752
22 16399-0,2025-06-10T10:05:03.901Z,44.940109,0.000000,29.746876
23 96-0,2025-06-10T10:05:04.900Z,44.984039,-0.000302,26.868752
24 16399-0,2025-06-10T10:05:04.901Z,44.940109,0.000000,29.746876
25 96-0,2025-06-10T10:05:05.900Z,44.984039,-0.000302,26.868752
26 16399-0,2025-06-10T10:05:05.901Z,44.940109,0.000000,29.746876
```

Durante la registrazione, in tempo reale, si può vedere anche il nome del file, quanto spazio di memoria sta occupando e da quanto tempo sta registrando il file in questione, così da avere una panoramica aggiuntiva.



Per quanto riguarda i dati dell'istogramma non c'è una registrazione in tempo reale dei dati, ma un download a fine acquisizione, per evitare continue sovrascritture del file.



Questa organizzazione rende i file pienamente compatibili con software di analisi esterni, come Excel e Python o strumenti scientifici personalizzati.

IV.IV.VII.II.III.I – Accesso al file system

L'accesso al file system in ZEUS è necessario per la gestione delle esportazioni.

I browser moderni impongono limitazioni all'accesso diretto al file system, per garantire la sicurezza dell'utente, ciò significa che il frontend, da solo, può solo iniziare i download o gestire file generati internamente, ma non può navigare nel file system.

Per superare queste limitazioni, Wails mette a disposizione delle funzionalità avanzate per superare queste barriere, consentendo di mostrare finestre di dialogo native del sistema operativo richiamando dal frontend funzioni Go esposte nel backend.

```
func (a *App) ChooseSaveDirectory() (string, error) {
    dirPath, err := runtime.OpenDirectoryDialog(a.ctx, runtime.OpenDialogOptions{
        Title: "Select the save folder",
    })
    if err != nil {
        return "", err
    }
    return dirPath, nil
}
```

Il codice Go sfrutta le API di Wails, che consentono di aprire una finestra di dialogo nativa per selezionare la cartella desiderata e restituisce al frontend il percorso selezionato.

Sul lato del frontend, React interagisce con questa funzione Go tramite il bridge di Wails, utilizzando l'oggetto `window.go.app`. Il codice React esegue la chiamata asincrona e il percorso scelto viene subito reso disponibile per salvare la cartella.

```
<i
  className='bx bxs-folder-open'
  style={{ cursor: "pointer" }}
  onClick={async () => {
    console.log("folderPath: ", saveDirectoryHistogram);
    try {
      const folderPath = await window.go.app.App.ChooseSaveDirectory();
      if (folderPath) {
        setSaveDirectoryHistogram(folderPath);
        console.log("📁 Cartella selezionata:", folderPath);
      }
    } catch (err) {
      console.error("❌ Errore selezione cartella:", err);
    }
  }}
></i>
```

Il bridge di Wails gestisce i privilegi di accesso in modo controllato, senza esporre il frontend a rischi.

IV.IV.VII.III – Curva I-V

Un componente avanzato di Zeus è la pagina dedicata alla scansione della curva I-V.

La curva I-V (corrente-tensione) è uno strumento fondamentale in elettronica e fisica, impiegato per studiare la risposta di un componente o di un dispositivo alla tensione applicata, osservando come varia la corrente che lo attraversa.

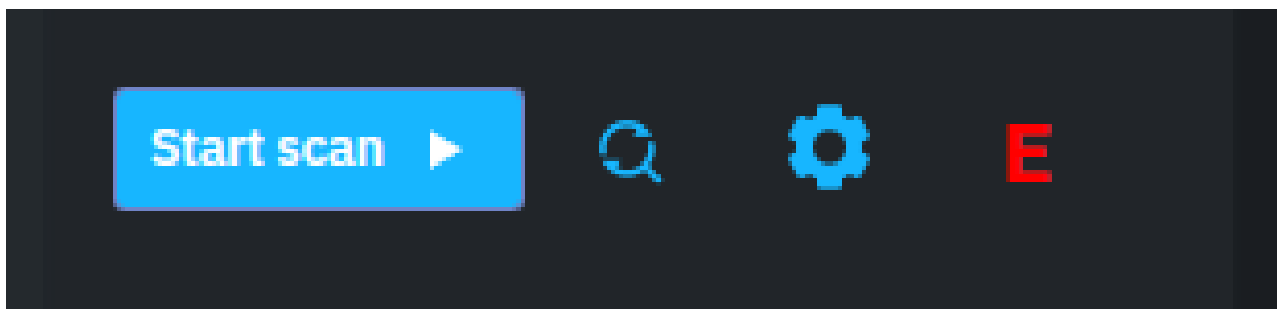
La curva I-V mette in relazione, in forma grafica, la tensione applicata al dispositivo e la corrente che scorre attraverso di lui. Ogni punto rappresentato sulla curva corrisponde a una coppia di valori (V, I), che tracciano l'andamento complessivo del dispositivo.



IV.IV.VII.III.I – Sidebar settings

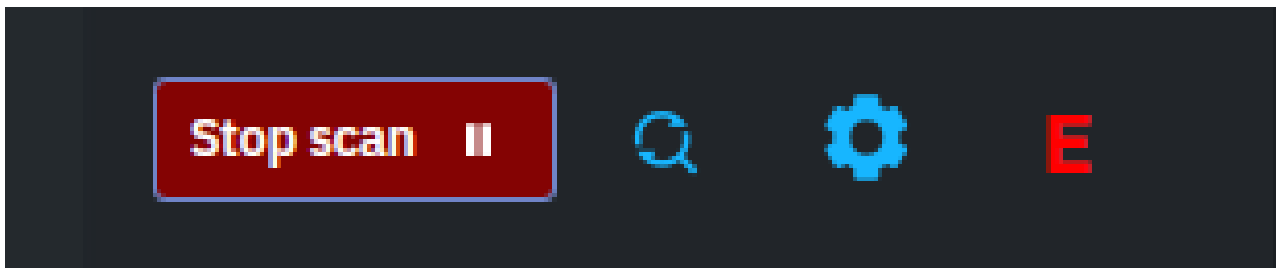
La Sidebar Settings rappresenta la configurazione avanzata delle scansioni della curva IV, essenziale per personalizzare nel dettaglio il comportamento dell'esperimento.

I pulsanti principali, posizionati nella parte superiore, sono composti da quattro pulsanti principali.

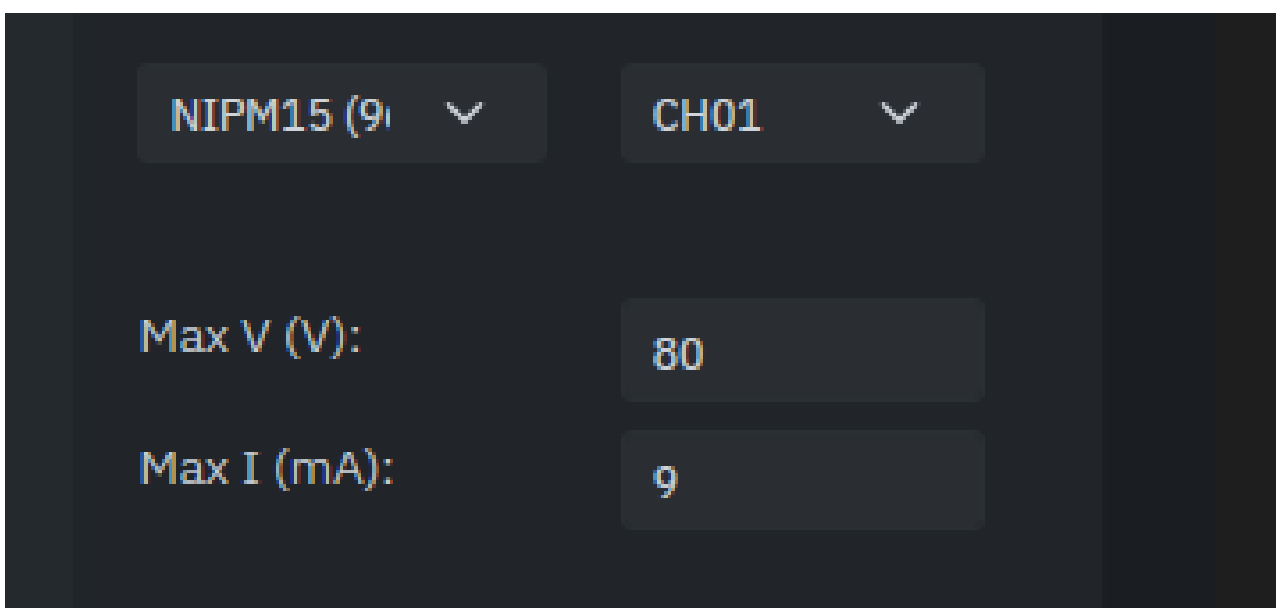


- Start scan, ha il compito di avviare la procedura di acquisizione della curva IV. Non appena viene premuto, il sistema si occupa automaticamente di accendere il dispositivo, se spento, e di portare la tensione al valore di partenza stabilito, per poi iniziare la scansione.
- Resize, ha lo scopo di ripristinare la scala e la vista predefinita dei grafici.
- Settings, apre un modal dedicato alla configurazione del download della curva.

- Emergency stop , è il pulsante d'emergenza, in caso di problemi, se premuto, interrompe immediatamente la scansione e spegne il dispositivo in questione.



Subito sotto i pulsanti principali trovi i selettori a tendina per scegliere dispositivo e canale su cui far partire la scansione. Subito sotto sono riportati i campi Max V (V) e Max I (mA), che mostrano rispettivamente la tensione massima e la corrente massima consentite per il canale selezionato, dati presenti anche nella tabella principale di Zeus.



I parametri della scansione rappresentano la parte essenziale della configurazione.

Start (V):	20
Stop (V):	72
Step (V):	1
Readings (n):	5

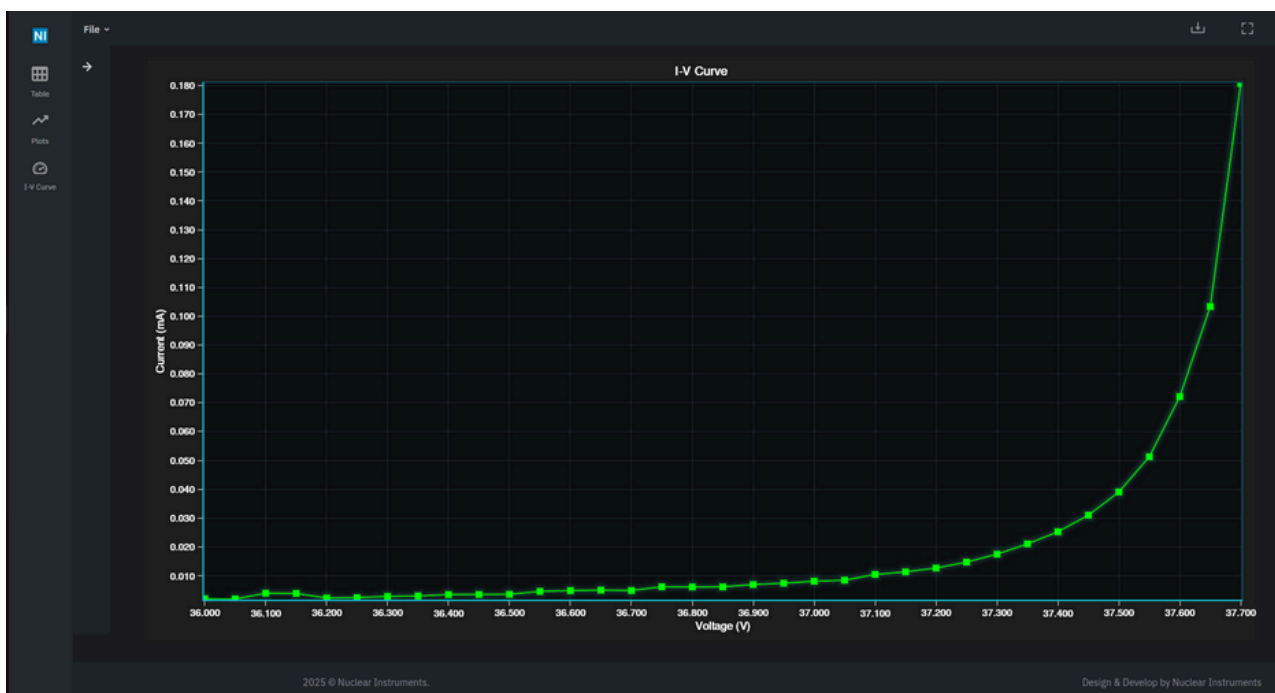
Current (mA)

Il campo Start (V) indica la tensione di partenza della scansione, Il parametro Stop (V) rappresenta invece la tensione di arresto, mentre lo Step (V) definisce l'incremento di tensione a ogni passo della curva. Infine, il parametro Readings (n) consente di indicare il numero di misure medie o acquisizioni da effettuare per ciascun punto.

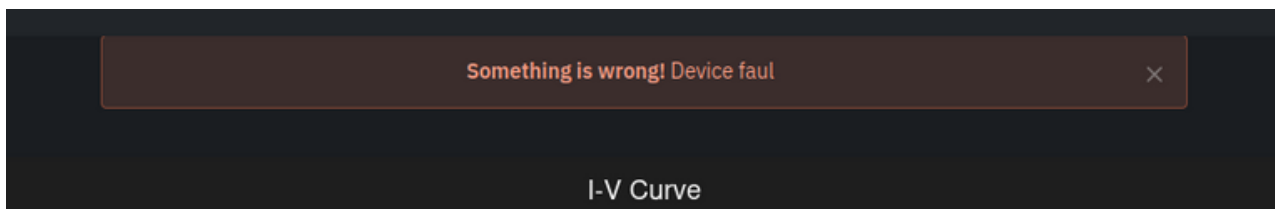
IV.IV.VII.III.II – Plot IV

Questo tipo di grafico rappresenta l'andamento della corrente elettrica in funzione della tensione applicata, fornendo informazioni essenziali sul comportamento elettrico del dispositivo sotto test. È ampiamente utilizzata per ottimizzare punti di lavoro e diagnosticare guasti.

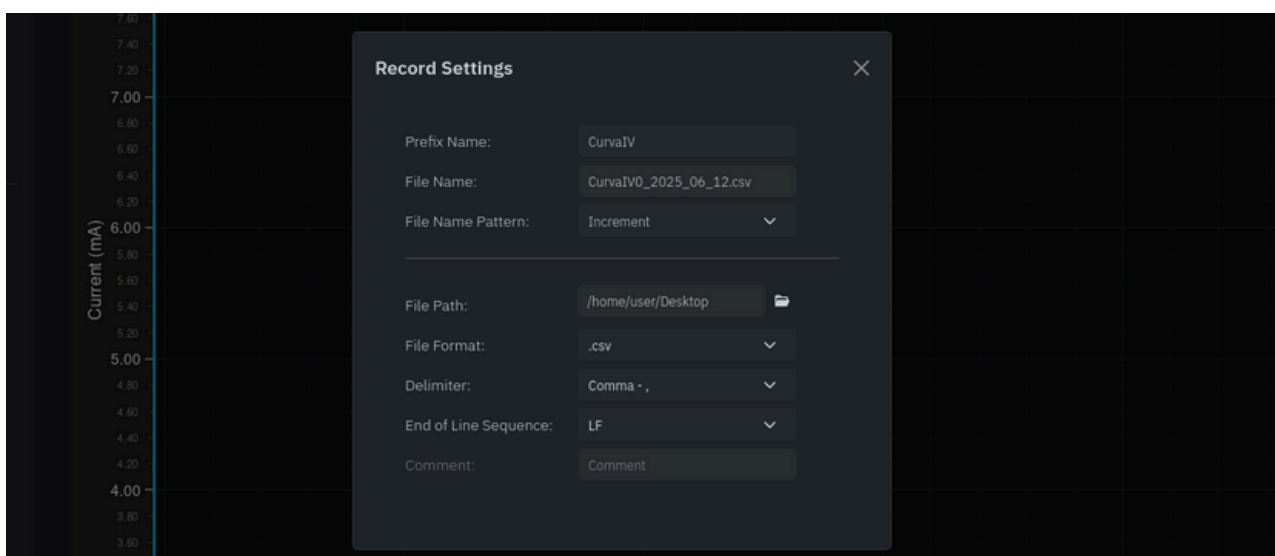
La Curva I-V è utilizzata per calcolare la VBR (tensione di break down del diodo), che è la tensione a cui si va ad operare per far funzionare correttamente un rilevatore SiPM (Silicon Photomultiplier), sensori di luce allo stato solido basati su silicio.



Durante l'acquisizione, ZEUS implementa un controllo automatico sullo stato del dispositivo. Se durante la scansione viene rilevata una condizione di fault (ad esempio superamento di limiti di corrente), il sistema interrompe immediatamente l'acquisizione. Questo meccanismo di protezione è fondamentale per garantire l'integrità sia del dispositivo sia del sensore collegato.



Anche qui, i dati raccolti durante la scansione possono essere salvati in un file .csv con, eventualmente, un header apposito. La configurazione può essere personalizzata tramite il pannello dedicato ("Record Settings").



V. CONCLUSIONI

V.I – Difficoltà incontrate

L'inizio della mia esperienza alla Nuclear Instruments è stato accompagnato da un mix di entusiasmo e timore. L'ambiente altamente qualificato, popolato da ingegneri e fisici con grande esperienza, unito all'alto livello dei progetti in corso, mi ha inizialmente intimorito.

Il progetto che mi è stato assegnato, ovvero la completa riscrittura e modernizzazione del software ZEUS, rappresentava una sfida che sin da subito, sembrava fuori dalla mia portata.

Non avevo mai lavorato su un progetto grosso e pensavo che le mie competenze non fossero all'altezza per progettare il tutto, dall'interfaccia grafica alla comunicazione con l'hardware, insieme all'integrazione di tecnologie avanzate come LightningChart o la gestione multi-piattaforma. Erano tutti elementi per me nuovi o solo parzialmente conosciuti.

Il carico di informazioni da gestire, insieme alla complessità della materia, mi hanno messo alla prova fin dai primi giorni.

Tuttavia, proprio queste difficoltà iniziali hanno rappresentato uno dei passaggi più formativi dell'intera esperienza. Ho imparato a non farmi bloccare dal timore iniziale, affrontando ogni problema per step.

Col tempo, grazie anche al supporto dei miei colleghi, sono riuscito a superare queste difficoltà e ad acquisire una nuova consapevolezza delle mie capacità.

V.II – Competenze ottenute

Durante lo sviluppo di ZEUS e grazie al contesto altamente professionale, ho avuto l'opportunità di acquisire un vasto insieme di competenze, che hanno significativamente arricchito il mio profilo professionale.

Sul piano tecnico, ho consolidato e ampliato le mie conoscenze nella programmazione orientata agli oggetti e allo sviluppo di interfacce web, applicando concretamente concetti che fino ad allora avevo affrontato solo in ambito accademico.

Ho sviluppato l'intero software adottando Go per il backend (linguaggio di programmazione totalmente nuovo e mai usato in ambito scolastico) e React per il frontend, approfondendo l'uso di framework moderni e strumenti di sviluppo professionali.

Ho imparato a lavorare con architetture modulari, a strutturare un progetto complesso in sottosistemi riutilizzabili e a gestire la comunicazione tra frontend e backend in modo efficiente.

Ho inoltre acquisito competenze pratiche nella gestione della comunicazione hardware/software, inclusa l'interazione con dispositivi.

Sul piano del design dell'interfaccia ero più preparato, ma ho imparato a tenere conto delle esigenze operative che un fisico può avere, grazie soprattutto ai colleghi che mi sono stati di aiuto.

In definitiva, questa esperienza ha rappresentato per me un salto di qualità, trasformando le competenze teoriche apprese nel percorso di studi, in abilità concrete spendibili nel mondo del lavoro.

V.III – Aspettative e realtà

Quando ho iniziato il tirocinio, le mie aspettative erano decisamente più contenute rispetto a ciò che si è poi realizzato.

Mi aspettavo, come spesso accade, di essere coinvolto in attività marginali, limitate a semplici esercizi. Non avrei mai immaginato che mi potessero affidare un progetto di questo tipo.

La realtà si è rivelata ben diversa, decisamente migliore. Fin dal primo giorno, sono stato immerso in un contesto altamente professionale, dove mi è stato chiesto di contribuire in modo attivo allo sviluppo di un software completo, pensato per essere utilizzato realmente in contesti di ricerca. Invece di svolgere compiti didattici, mi sono ritrovato a sviluppare ZEUS, un'applicazione multiplatforma, in grado di interfacciarsi con strumentazione hardware reale.

Nonostante l'iniziale senso di spaesamento, ho trovato nell'ambiente di lavoro un forte supporto, fatto di persone giovani, competenti e soprattutto, cosa non scontata, disponibili.

Questo mi ha permesso di procedere, acquisendo sicurezza e fiducia in me stesso.

A poco a poco, ho scoperto di essere in grado di realizzare molto più di quanto pensassi, e di poter contribuire davvero a un progetto reale e innovativo.

Questa esperienza ha superato di gran lunga ogni mia aspettativa, trasformandosi in un momento di crescita professionale e personale.

Zeus attualmente è in validazione presso il reparto qualità. Dopo di che verrà scritto il manuale e si prevede il rilascio al pubblico verso settembre con il nome di Zeus 2, rimpiazzando l'attuale versione di Zeus che non sarà più mantenuta.

VI. SITOGRAFIA

- <https://www.nuclearinstruments.eu/>
- <https://www.caen.it/>
- <https://www.caen.it/products/zeus-control-software/>
- <https://code.visualstudio.com/docs>
- <https://wails.io/docs/introduction>
- <https://go.dev/ref/spec>
- <https://react.dev/reference/react>
- <https://scarsi.web.cern.ch/>

VII. RINGRAZIAMENTI

Arrivato al termine di questo percorso, mi sembra doveroso ringraziare alcune delle persone che hanno permesso tutto ciò.

La mia famiglia per averci creduto anche quando, a volte, avevo smesso di farlo.

La mia tutor, Martina Lombardi e il prof. Romeo Mariani, per aver creduto che fossi il ragazzo giusto per questa esperienza.

Tutta la Nuclear Instruments per l'accoglienza, l'opportunità di crescita e soprattutto l'opportunità, da settembre, di essere assunto come dipendente dell'azienda.

Ringrazio in particolare il mio tutor aziendale Andrea Abba, per l'enorme fiducia, tutta la sala software, in particolare Valentina, sviluppatrice del primo Zeus, per le tecniche sui firmware degli HV. Francesco, compagno di sviluppo di interfacce grafiche per i preziosi consigli. Infine Stefano Carsi, fisico e programmatore FPGA, per l'enorme pazienza rivolta nei miei confronti per la spiegazione anche più basilare dei concetti elementari della fisica.

Grazie a tutti

LUCA RIGAMONTI